第13章　聚 类

13.1 什么是聚类

13.2 K-MEANS算法

13.3 聚类结果的评价

13.4 不同的距离指标

13.5 聚合式层次聚类

机器学习类型

有监督	数据点有已知的结果
无监督	数据点没有已知的结果

机器学习类型

有监督	数据点有已知的结果
无监督	数据点没有已知的结果

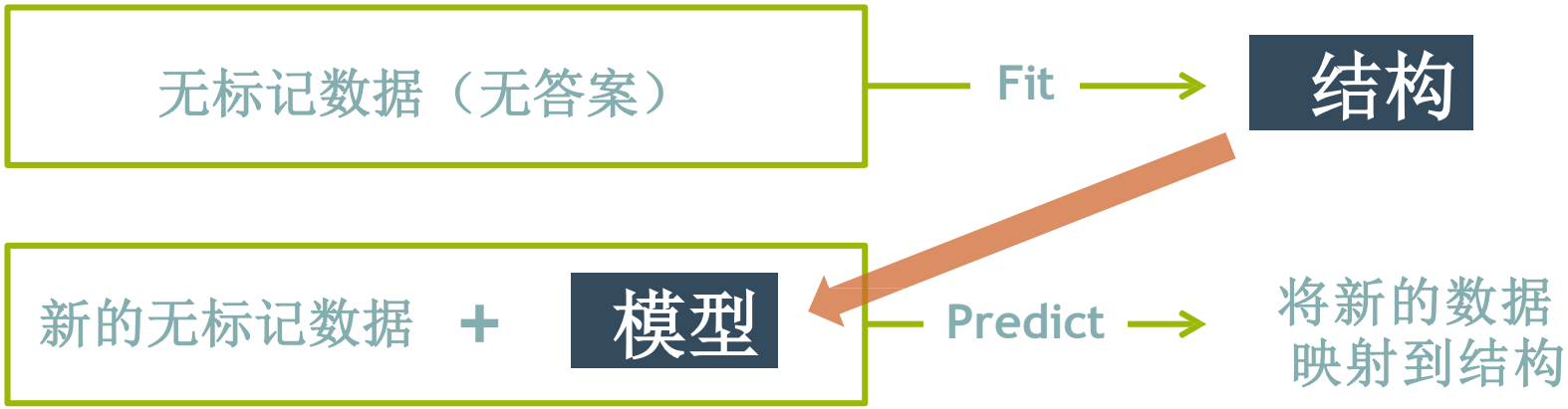
无监督学习的类型

聚类	识别数据中未知的结构

无监督学习的类型

聚类	识别数据中未知的结构
降维	使用数据中的结构特征来简化数据

无监督学习概述

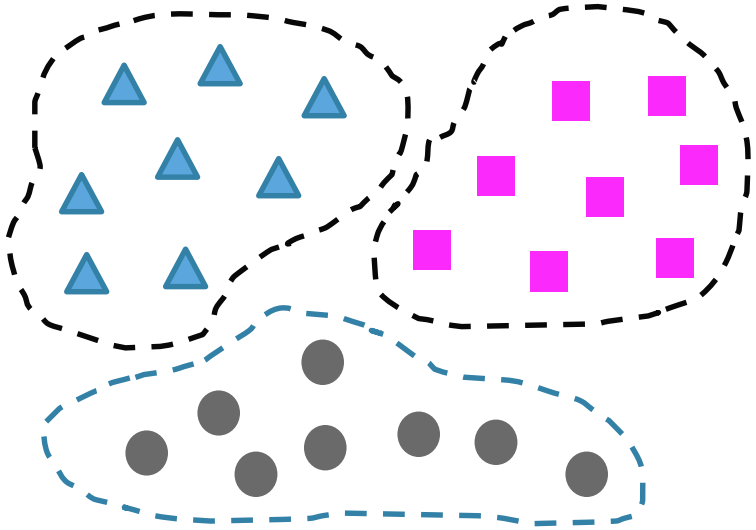


聚类任务

在“无监督学习”任务中研究最多、应用最广.

聚类目标：将数据集中的样本划分为若干个通常不相交的子集(“簇”，cluster).

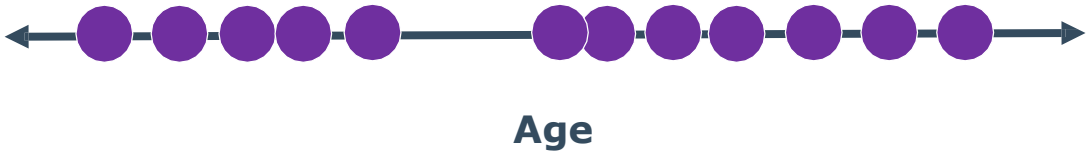
聚类既可以作为一个单独过程(用于找寻数据内在的分布结构)，也可作为分类等其他学习任务的前驱过程.



聚类

一个web应用的所有用户：

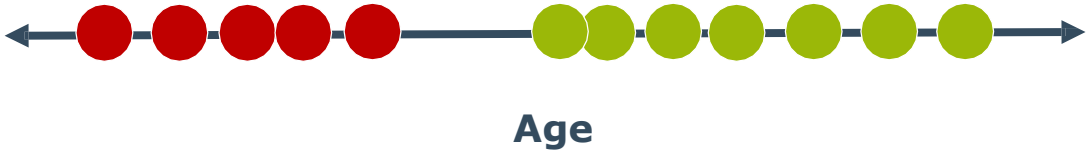
- 一个特征 (age)



聚类

一个web应用的所有用户：

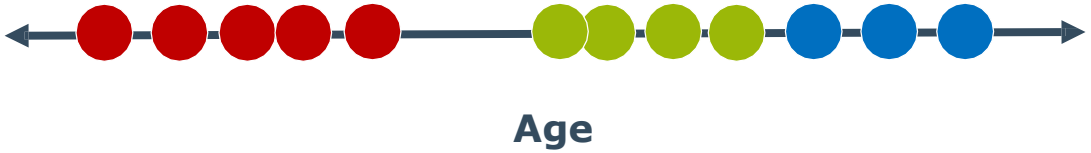
- 一个特征 (age)
- 两个聚簇 (cluster)



聚类

一个web应用的所有用户：

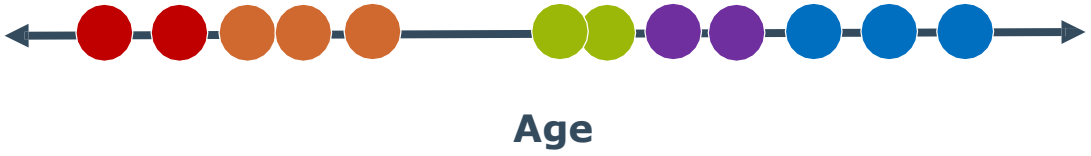
- 一个特征 (age)
- 三个聚簇 (cluster)

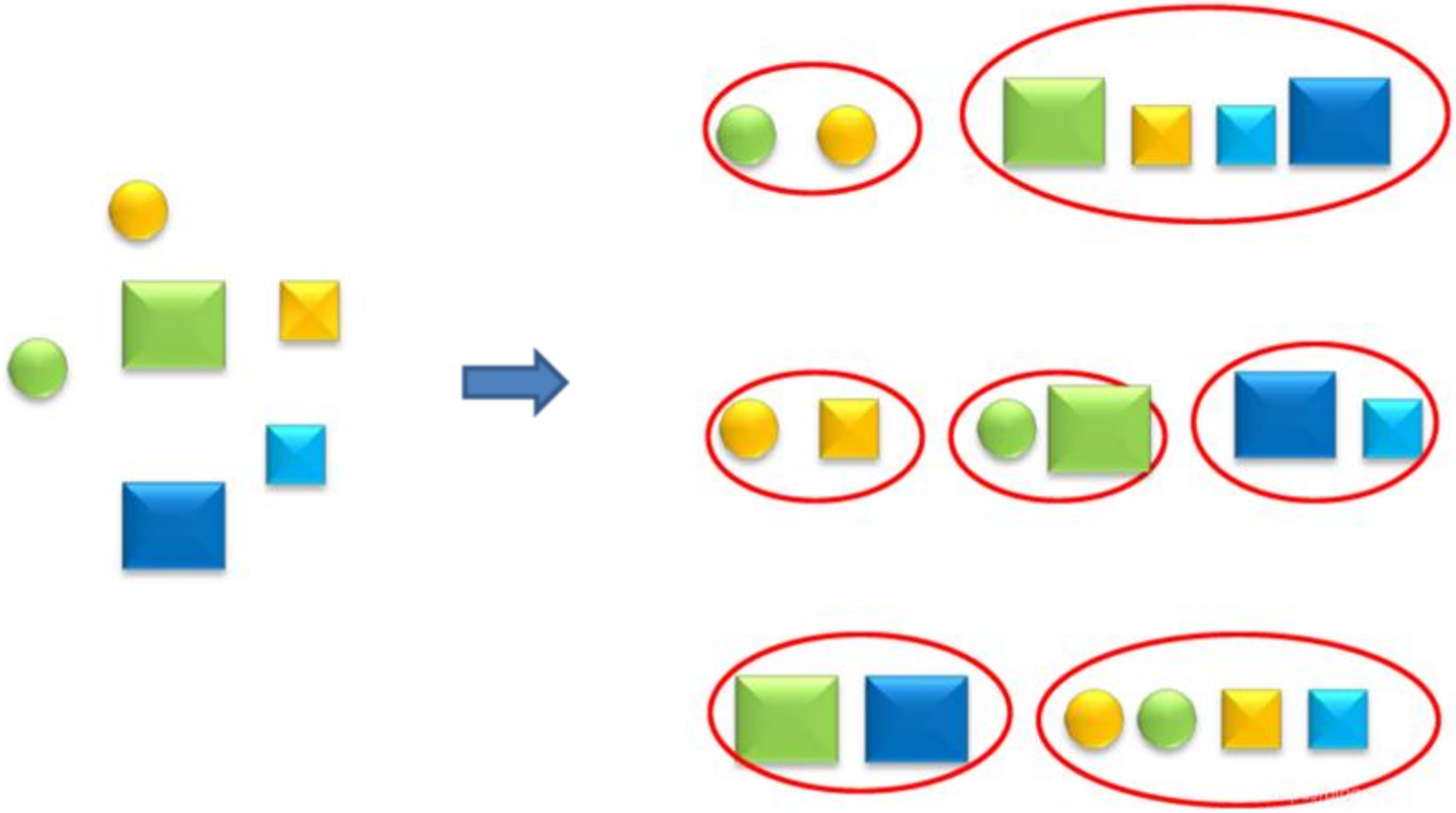


聚类

一个web应用的所有用户：

- 一个特征 (age)
- 五个聚簇 (cluster)

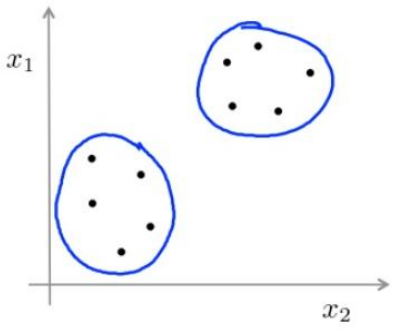


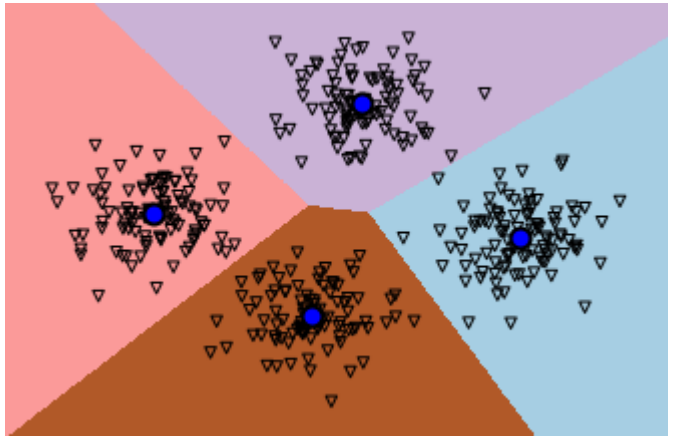


形式化描述

假定样本集$D=\{x_1,x_2,\cdots,x_m\}$包含m个无标记样本，每个样本$x_i=(x_{i1};x_{i2};\cdots;x_{in})$是一个$n$维的特征向量，聚类算法将样本集$D$划分成$k$个不相交的簇$\{C_l|l=1,2,...,k\}$。其中 $C_{l'}\bigcap_{l'\neq l}C_l=\phi$，且 $D=\bigcup_{l=1}^{k}C_l$ 。

相应地，用$\lambda=\{\lambda_1;\lambda_2;\cdots;\lambda_m\}$表示样本$x_j$的“簇标记”（即cluster label），即$x_j\in C_{\lambda_j}$。于是，聚类的结果可用包含$m$个元素的簇标记向量$\lambda_j\in\{1,2,\cdots,k\}$表示。





13.1 什么是聚类

13.2 K-MEANS算法

13.3 聚类结果的评价

13.4 不同的距离指标

13.5 聚合式层次聚类

k均值算法

给定数据集 $D=\{x_1,x_2,\cdots,x_m\}$ ， k均值算法针对聚类所得簇划分 $C=\{C_1,C_2,\cdots,C_k\}$ 最小化平方误差

$$E=\sum_{i=1}^{k}\sum_{x\in C_i}\|x-\mu_i\|_2^2$$

$$u_i=\frac{1}{|C_i|}\sum_{x\in C_i}x$$

其中，μ_i 是簇 C_i 的均值向量。

E 值在一定程度上刻画了簇内样本围绕簇均值向量的紧密程度，E 值越小，则簇内样本相似度越高。

- 算法流程（迭代优化）：

初始化每个簇的均值向量

repeat

1. （更新）簇划分；
2. 计算每个簇的均值向量

until 当前均值向量均未更新

k均值算法

输入: 样本集$D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$;
聚类簇数k.

过程:

```
从D中随机选择k个样本作为初始均值向量{μ_1, μ_2, ..., μ_k}
repeat
    令C_i = ∅ (1 ≤ i ≤ k)
    for j = 1, ..., m do
        计算样本x_j与各均值向量μ_i (1 ≤ i ≤ k)的距离: d_ji = ||x_j − μ_i||_2;
        根据距离最近的均值向量确定x_j的簇标记: λ_j = arg min_{i∈{1,2,...,k}} d_ji;
        将样本x_j划入相应的簇: C_{λ_j} = C_{λ_j} ⋃ {x_j};
    end for
    for i = 1, ..., k do
        计算新均值向量: μ'_i = (1/|C_i|) Σ_{x∈C_i} x;
        if μ'_i ≠ μ_i then
            将当前均值向量μ_i更新为μ'_i
        else
            保持当前均值向量不变
        end if
    end for
until 当前均值向量均未更新
return 簇划分结果
```

输出: 簇划分$\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$

k均值算法实例

以表中数据集为例，演示k均值算法的学习过程。将编号为 i 的样本称为 x_i .

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

k均值算法实例

假定聚类簇数k =3，算法开始时，随机选择3个样本 x_6, x_{12}, x_{27} 作为初始均值向量，即 $\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$

考察样本 $x_1 = (0.697; 0.460)$，它与当前均值向量 μ_1, μ_2, μ_3的距离分别为0.369，0.506，0.166，因此 x_1 将被划入簇 C_3 中。类似的，对数据集中的所有样本考察一遍后，可得当前簇划分为

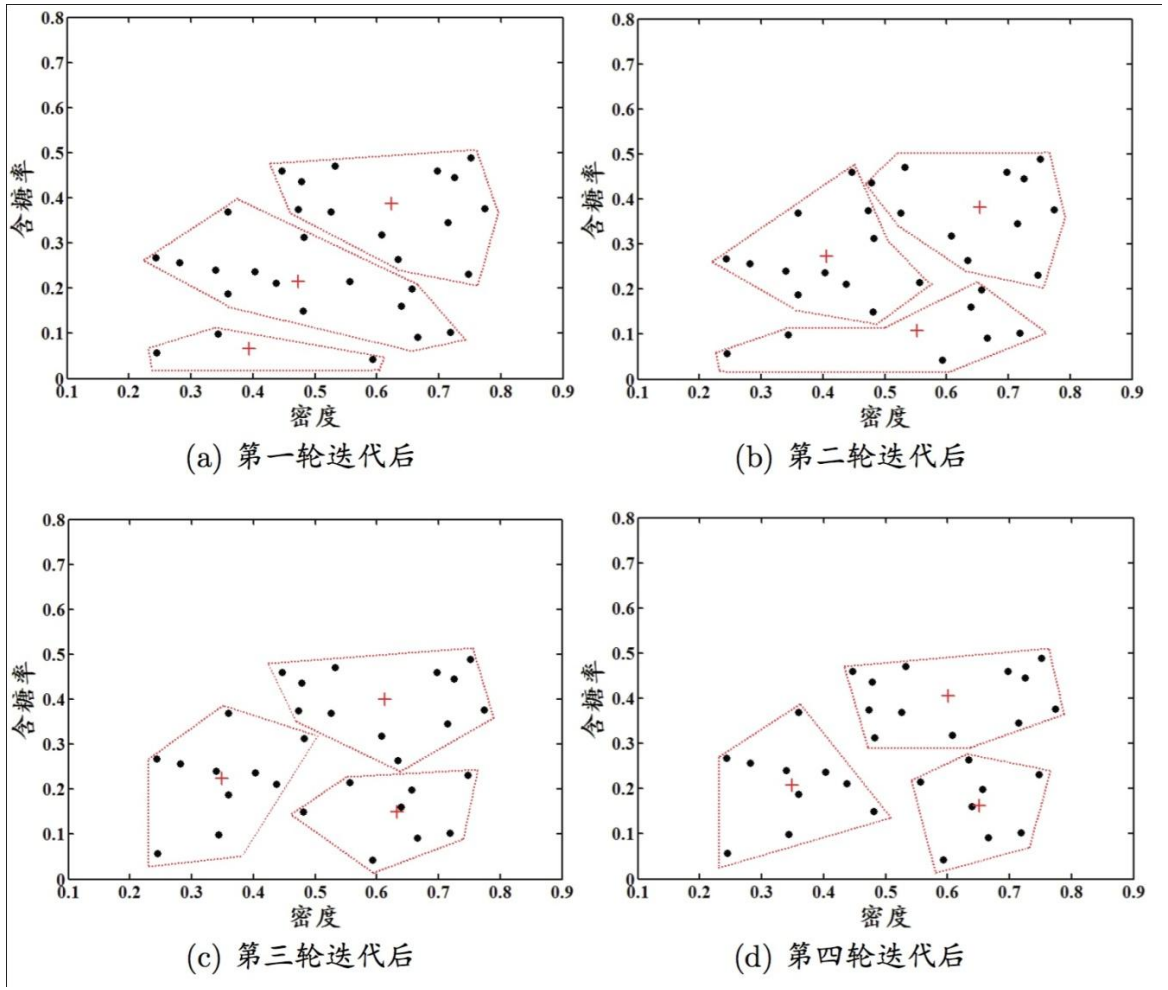
$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$
$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$
$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

于是，可以分别求得新的均值向量

$$\mu_1' = (0.473; 0.214), \mu_2' = (0.394; 0.066), \mu_3' = (0.623; 0.388)$$

不断重复上述过程。

聚类结果：



(a) 第一轮迭代后 (b) 第二轮迭代后

(c) 第三轮迭代后 (d) 第四轮迭代后

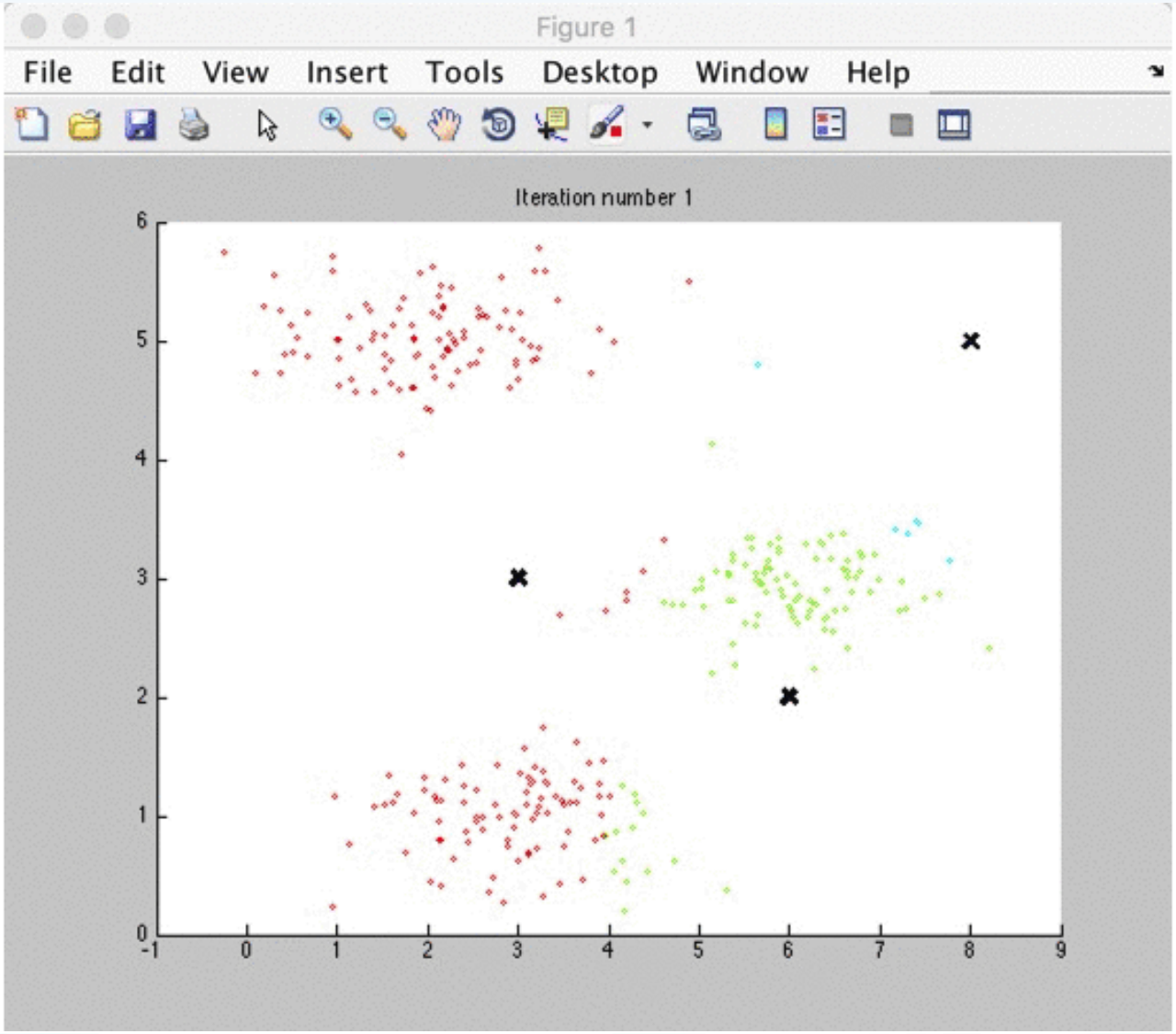
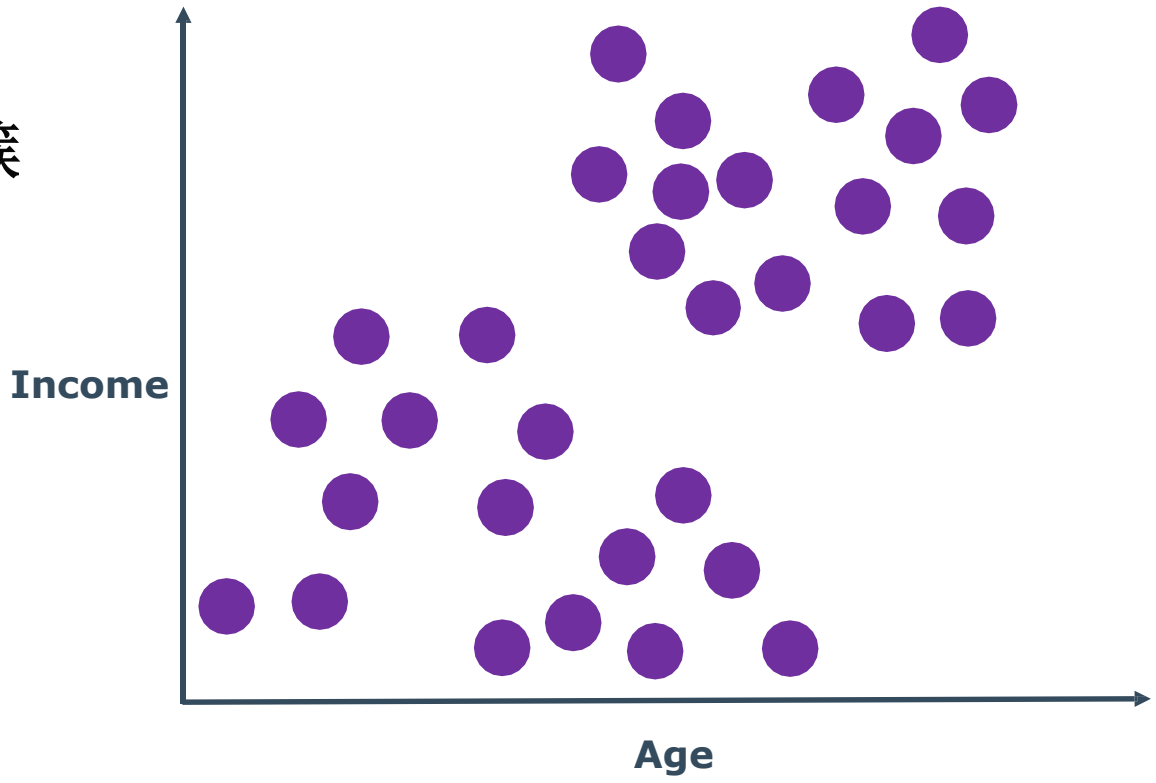


Figure 1
File Edit View Insert Tools Desktop Window Help
Iteration number 1
6
5
4
3
2
1
0
-1 0 1 2 3 4 5 6 7 8 9

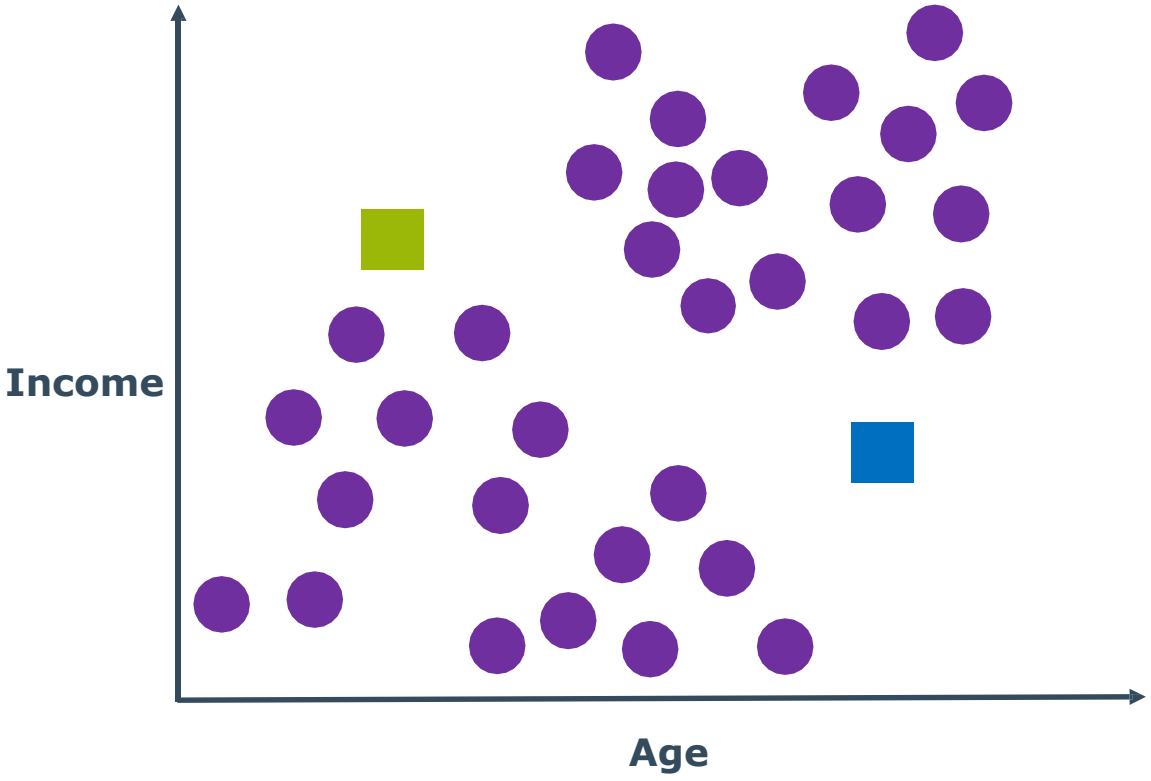
K-Means 算法

K = 2：发现两个聚簇



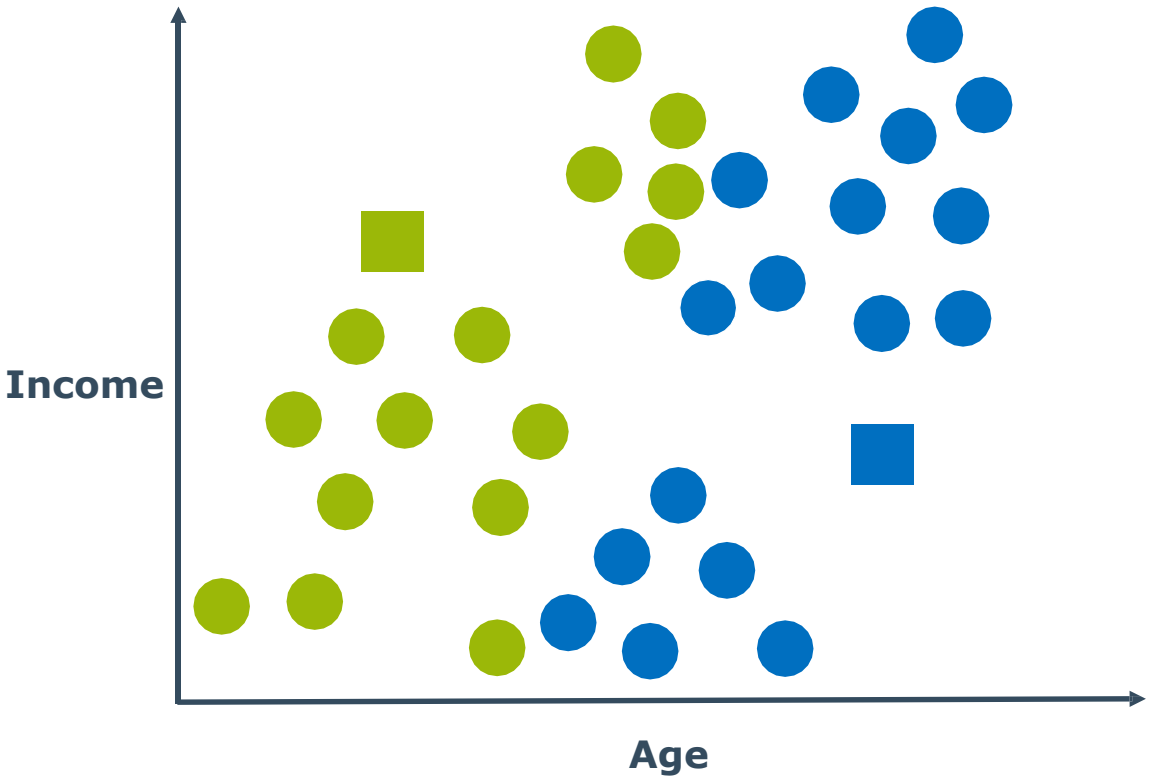
K-Means 算法

K = 2, 随机地选择两个聚簇的中心点



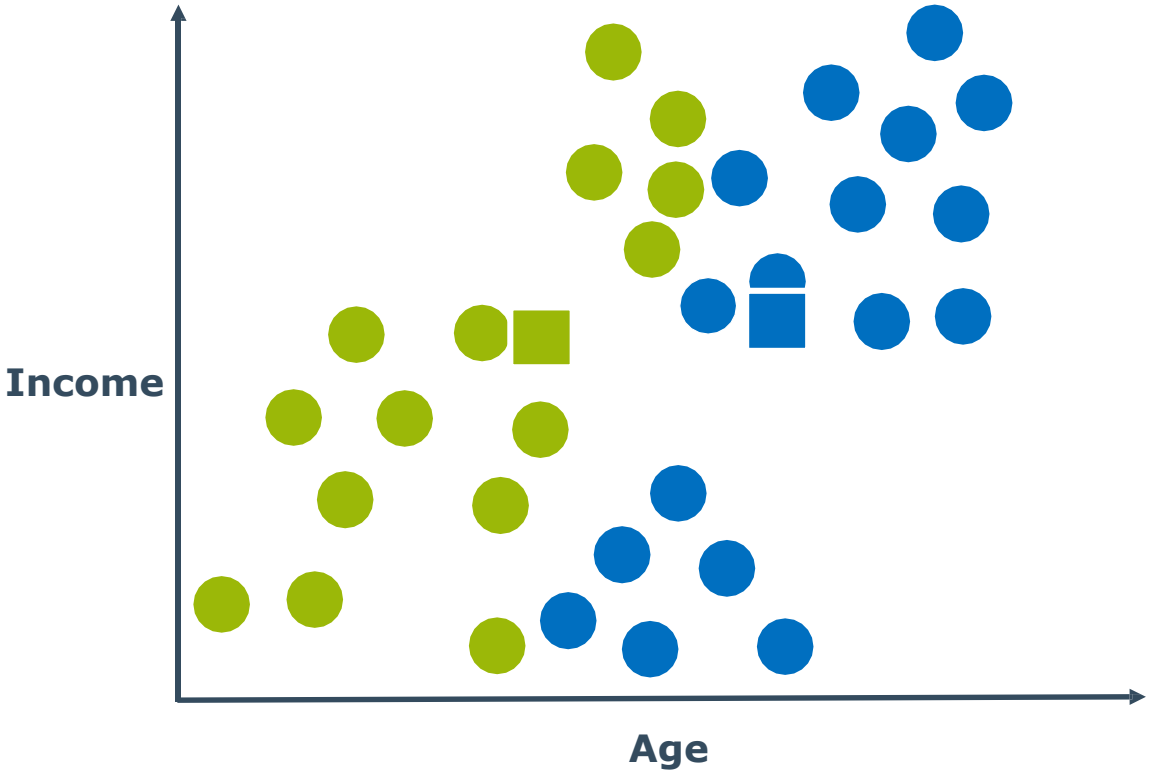
K-Means 算法

K = 2, 每个点被聚到离它最近的中心点代表的聚簇中



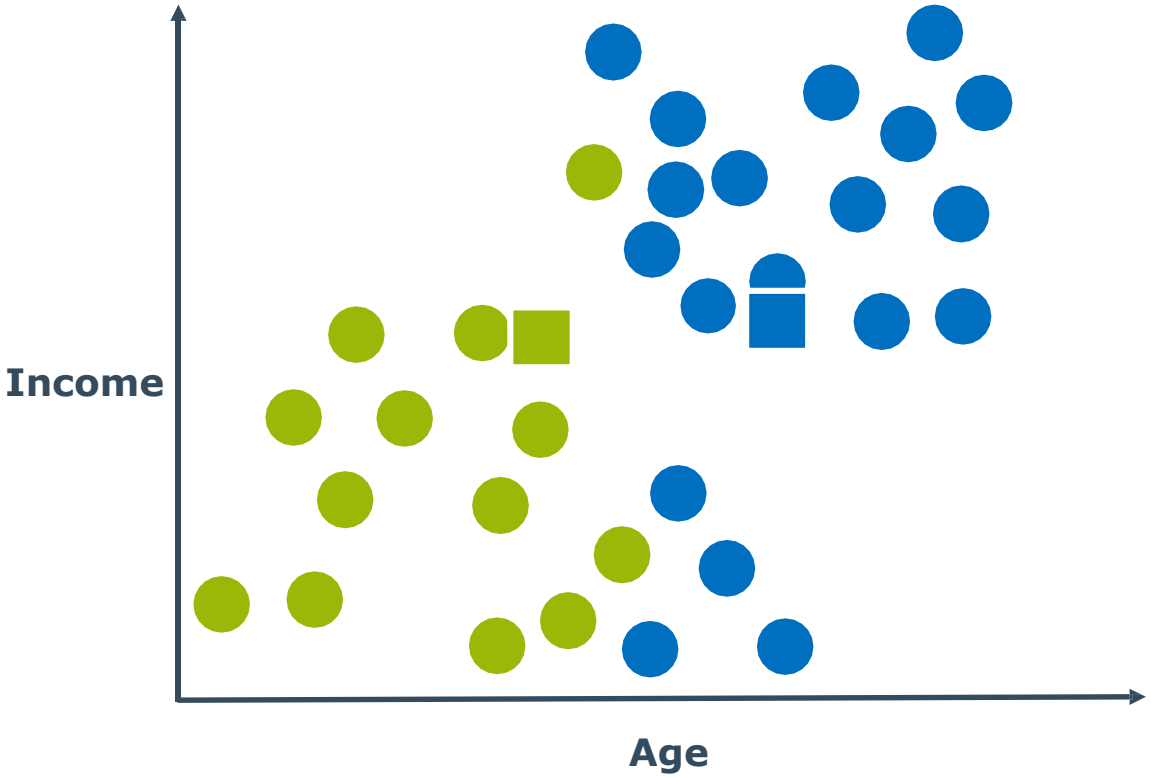
K-Means 算法

K = 2, 重新计算每个聚簇的中心点



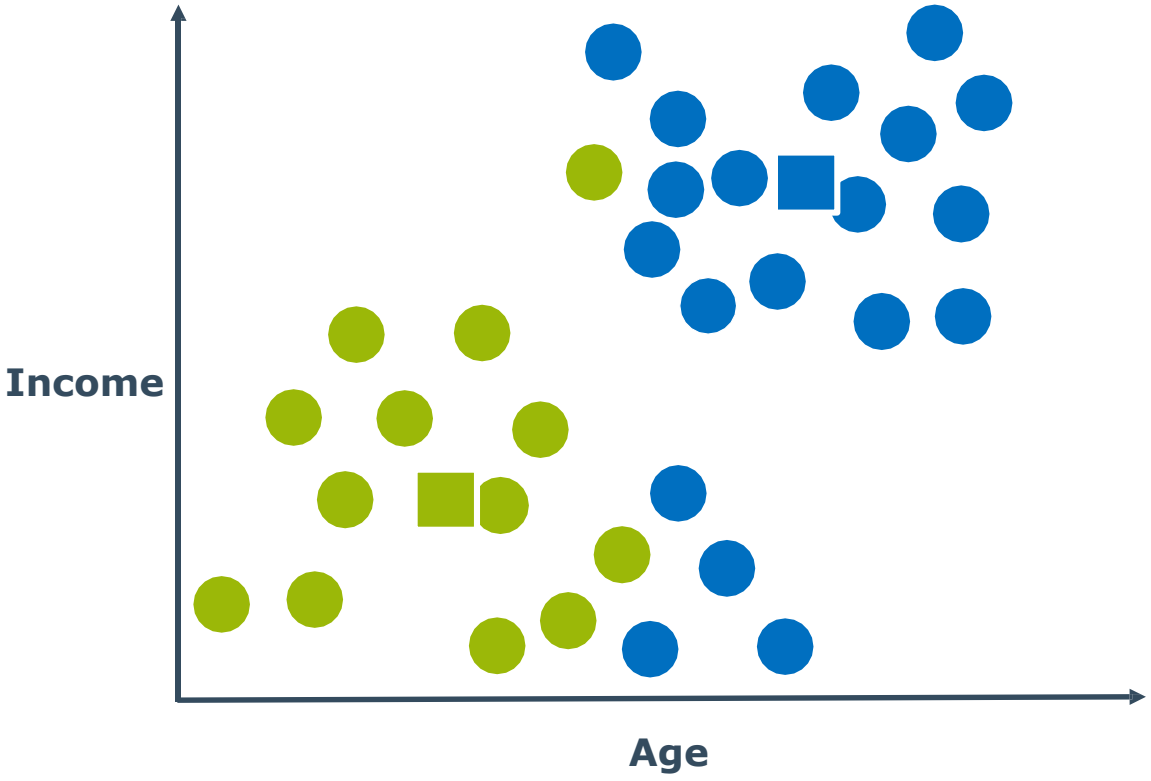
K-Means 算法

K = 2, 每个点属于离它最近的中心点



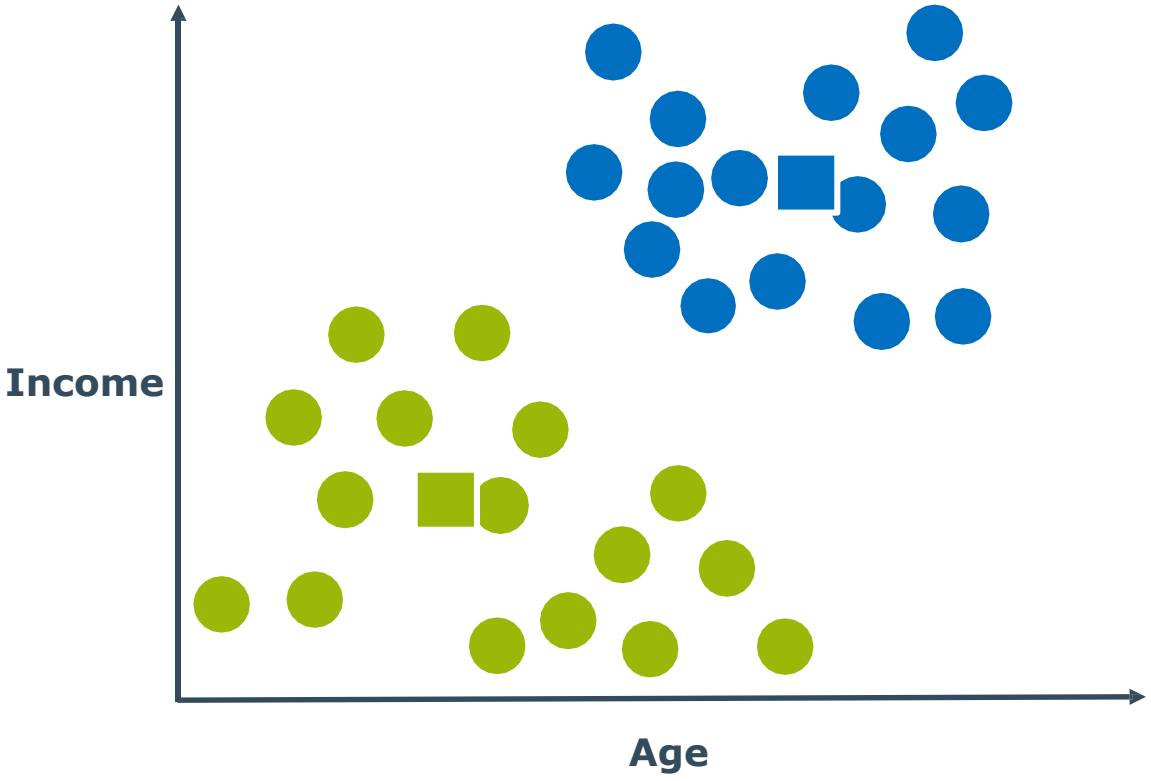
K-Means 算法

K = 2, 重新计算每个聚簇的中心点



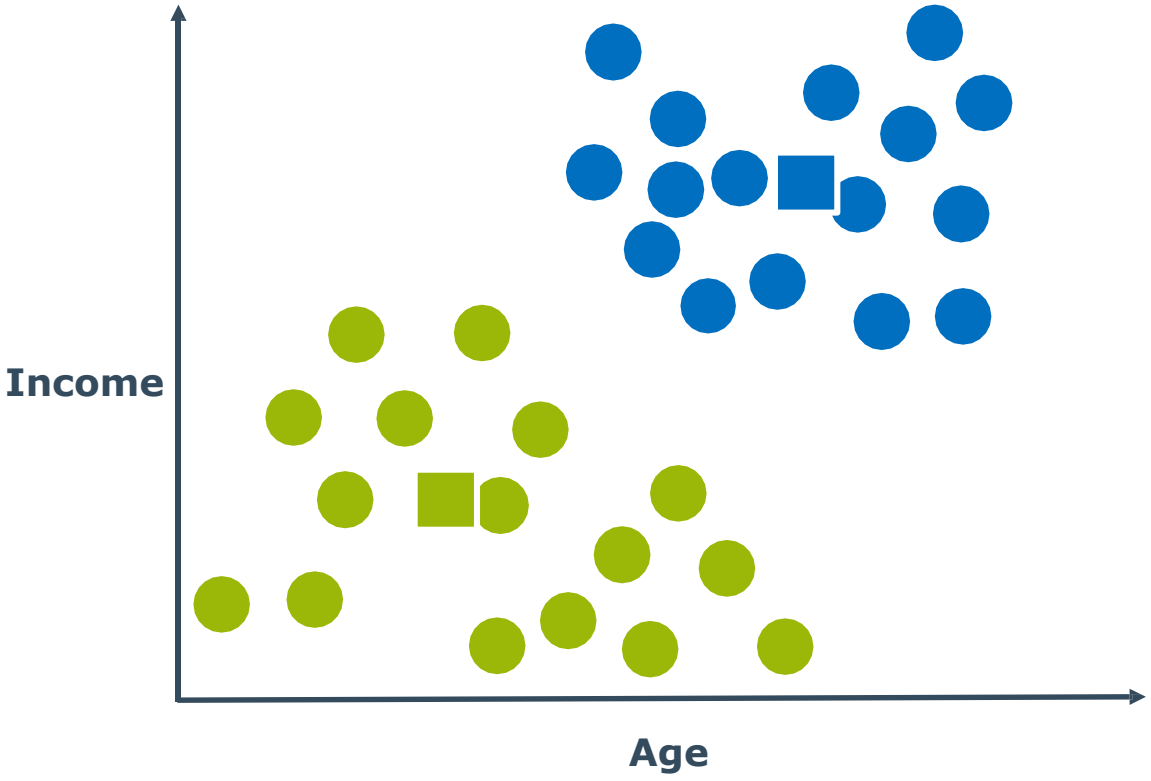
K-Means 算法

K = 2, 数据点的聚簇不再改变时→ 算法收敛



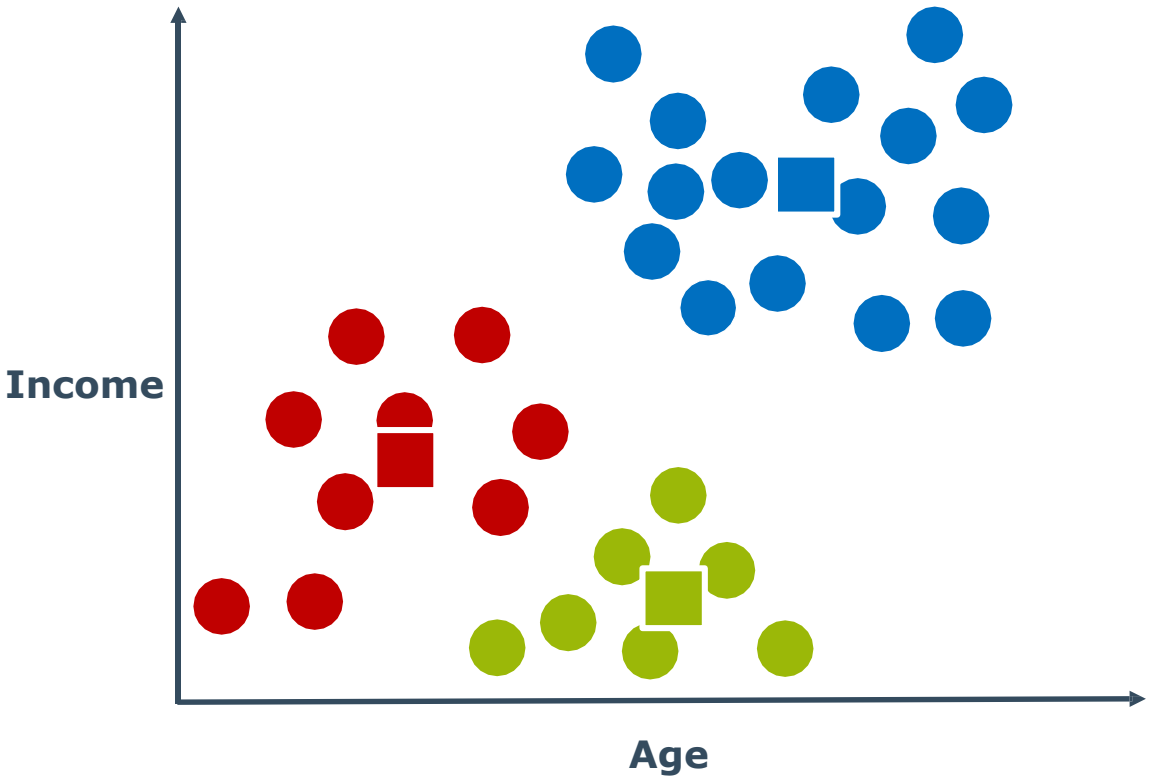
K-Means 算法

K = 2, 最后每个点属于离它最近的那个聚簇



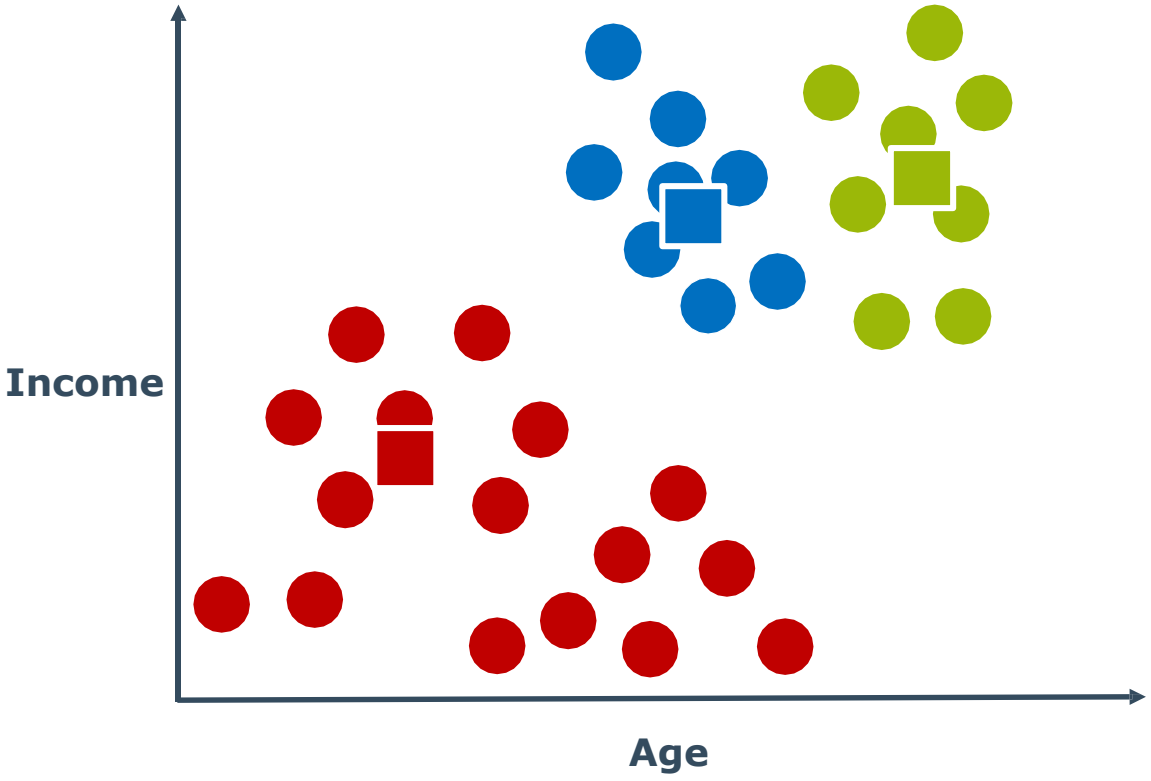
K-Means 算法

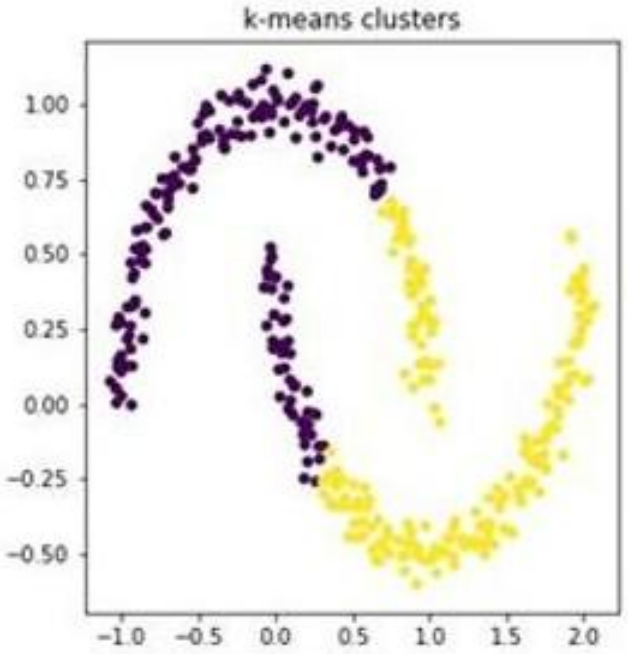
K = 3

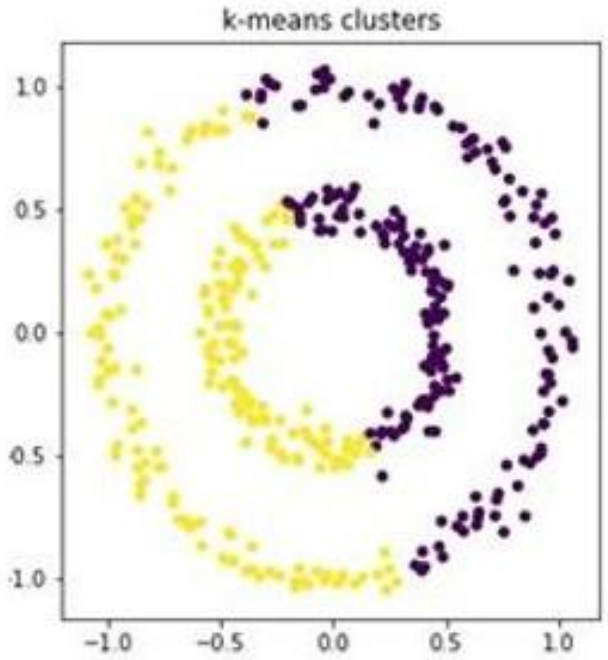


K-Means 算法

K = 3, 最终结果依赖于初始的聚簇分配




k-means clusters
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


k-means clusters
1.0
0.5
0.0
-0.5
-1.0
−1.0
−0.5
0.0
0.5
1.0

13.1 什么是聚类

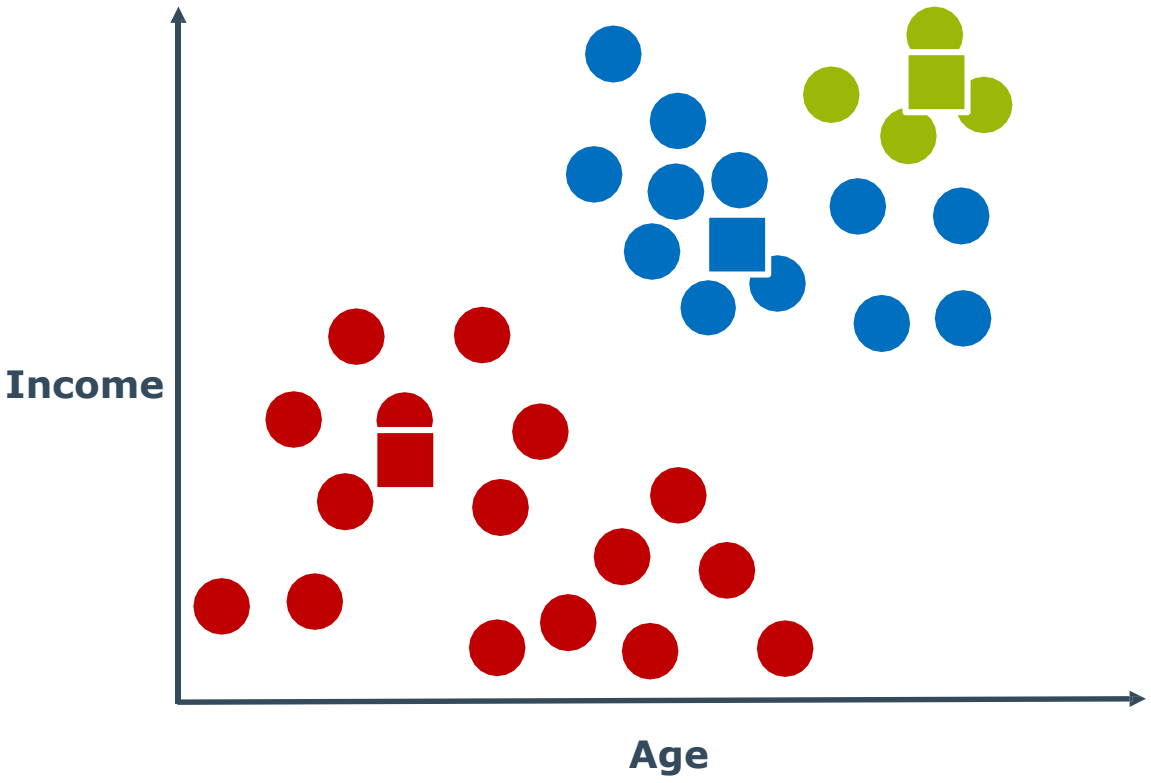
13.2 K-MEANS算法

13.3 聚类结果的评价

13.4 不同的距离指标

13.5 聚合式层次聚类

哪个模型正确？

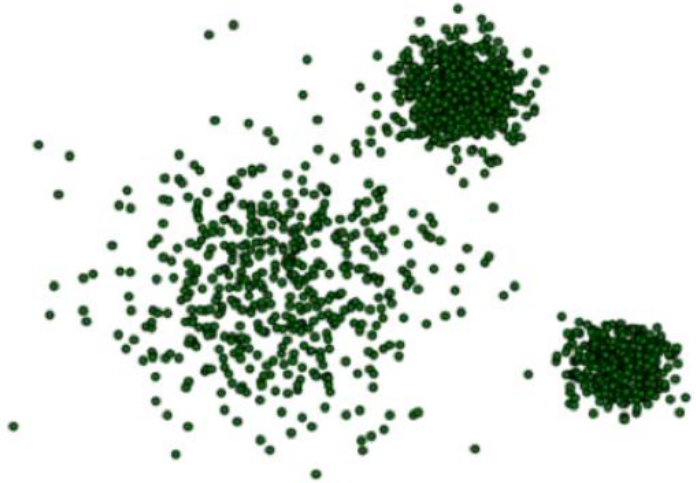


性能度量

聚类性能度量，亦称为聚类“有效性指标”(validity index)

直观来讲：

我们希望“物以类聚”，即同一簇的样本尽可能彼此相似，不同簇的样本尽可能不同。换言之，聚类结果的“簇内相似度”(intra-cluster similarity)高，且“簇间相似度”(inter-cluster similarity)低，这样的聚类效果较好.



聚类性能度量：

- 外部指标（external index）

  将聚类结果与某个“参考模型”(reference model)进行比较。

- 内部指标（internal index）

  直接考察聚类结果而不用任何参考模型。

聚类评价指标

- **Inertia（簇内平方和）：**
  - 每个数据点(x_i)距其聚簇中心(C_k)的距离平方和

$$\sum_{i=1}^{n}(x_i - C_k)^2$$

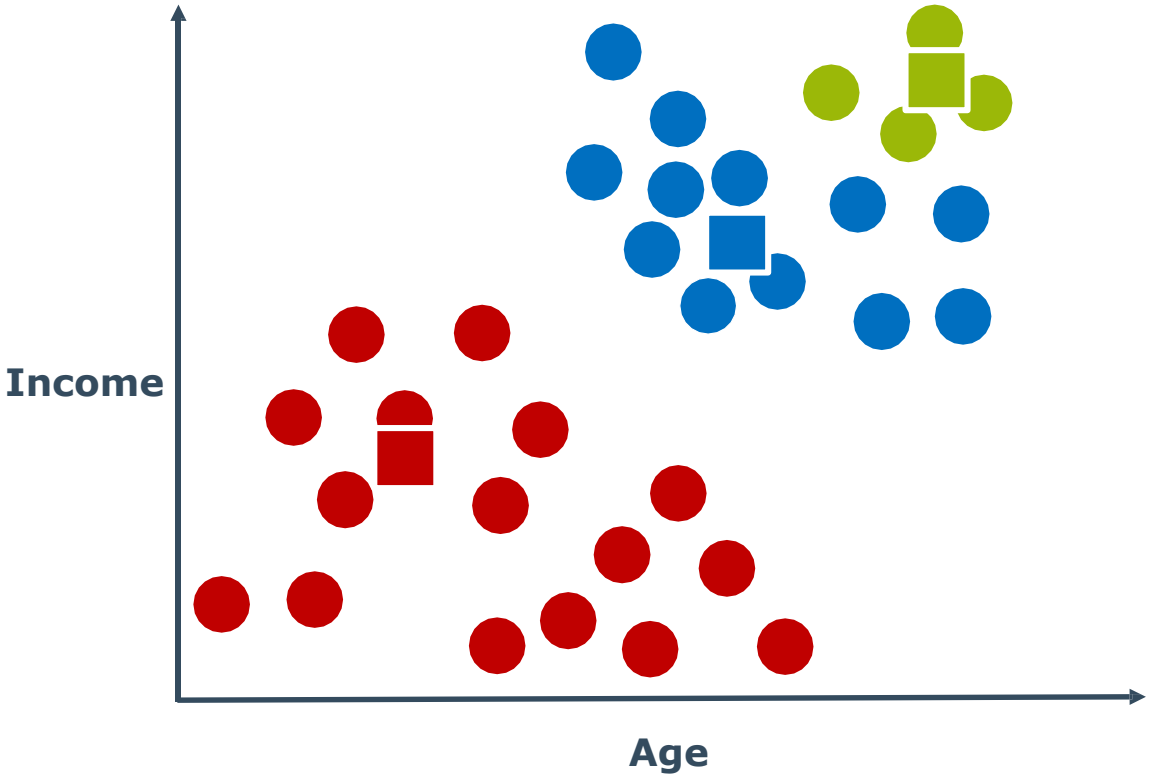
  - 值越小表示聚簇越紧密
- **轮廓系数（Silhouette Coefficient）：**
  - 对每个数据点计算一个轮廓系数：

$$S = \frac{b-a}{\max(a,b)}$$

    - a = 此数据点到**同簇**中所有其他点的平均距离，凝聚度
    - b = 此数据点到**最近簇**中所有点的平均距离，分离度
  - 将所有数据点的轮廓系数取平均值就得到一个总的评分
  - 取值在[-1, 1]之间，值越大，聚类效果越好

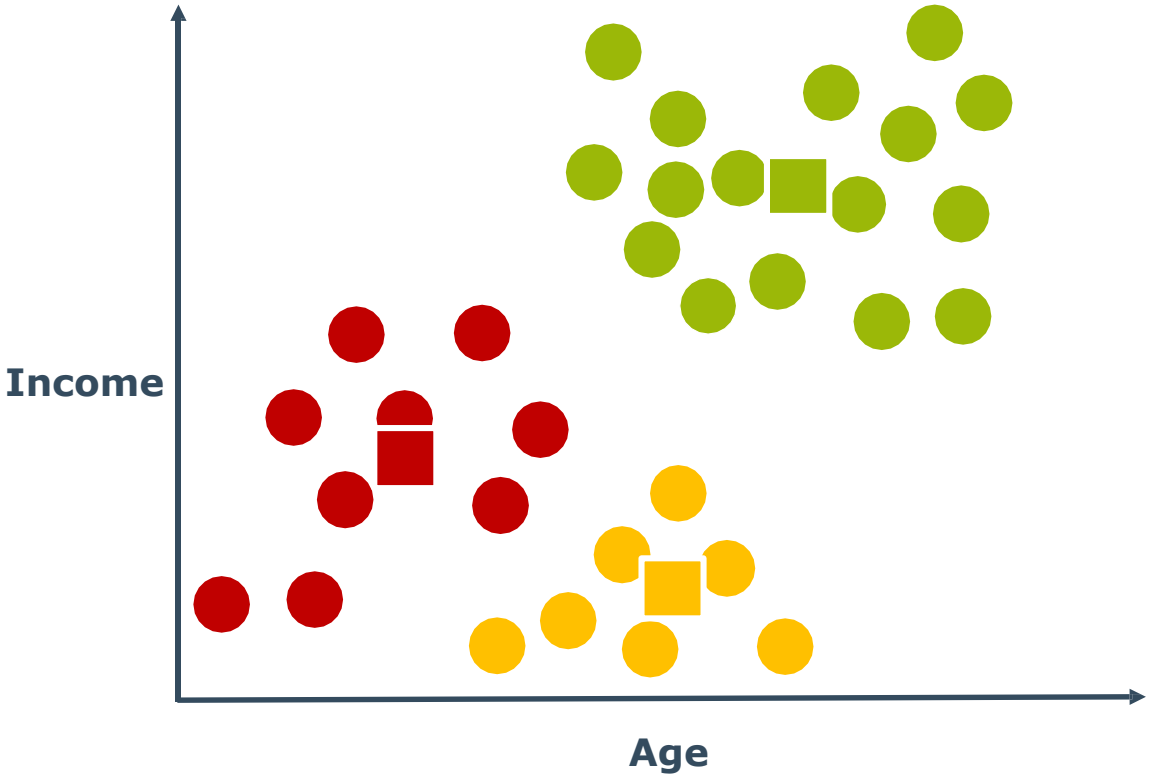
哪个模型正确？

初始化多次，然后选择得分最高的模型



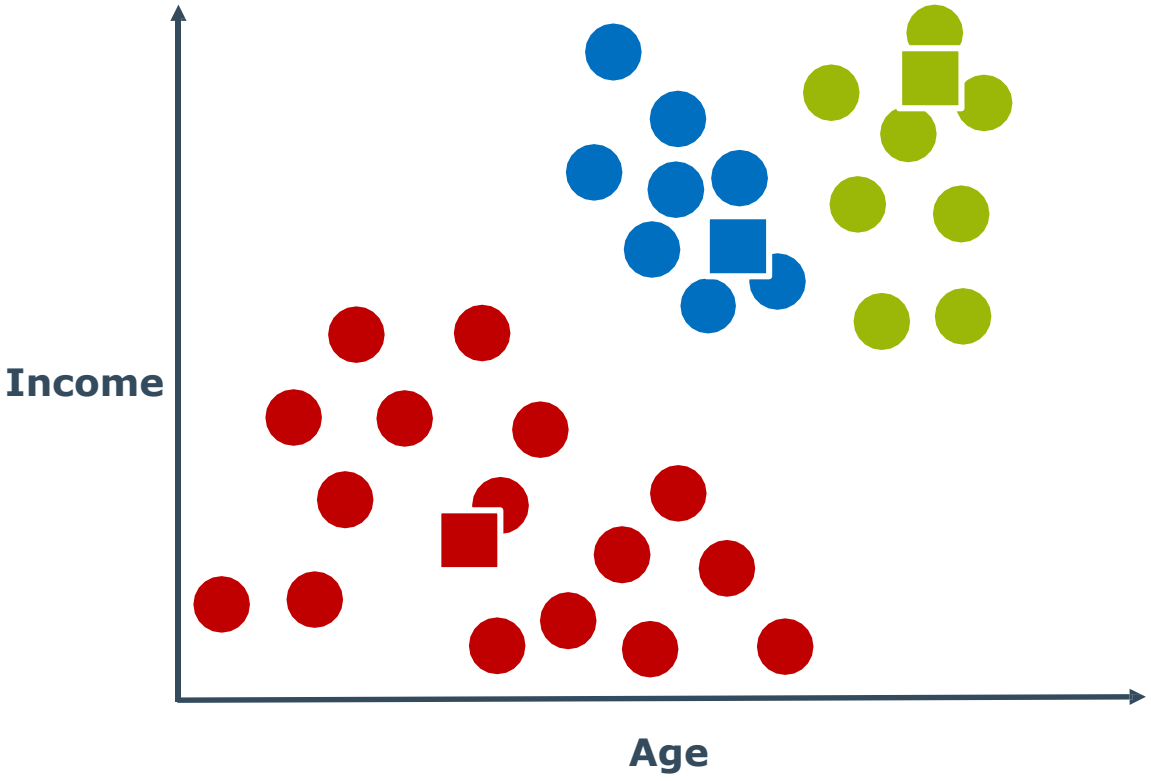
哪个模型正确？

Inertia = 12.645



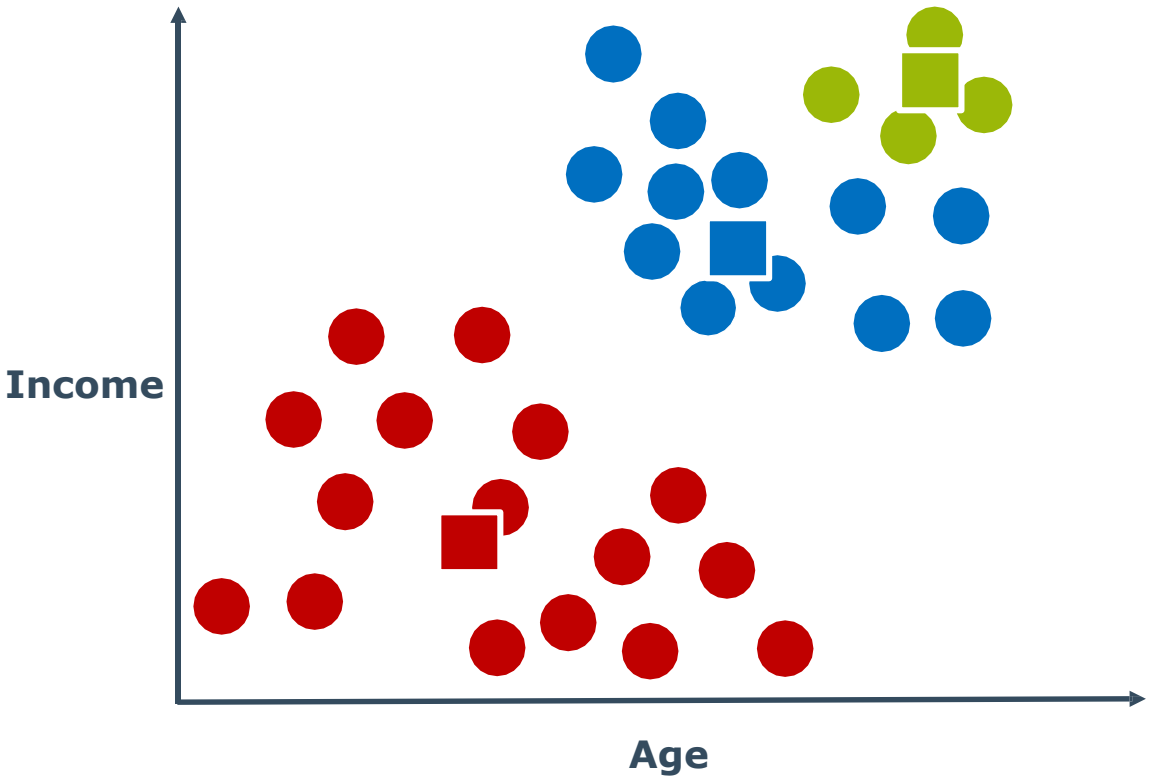
哪个模型正确？

Inertia = 12.943



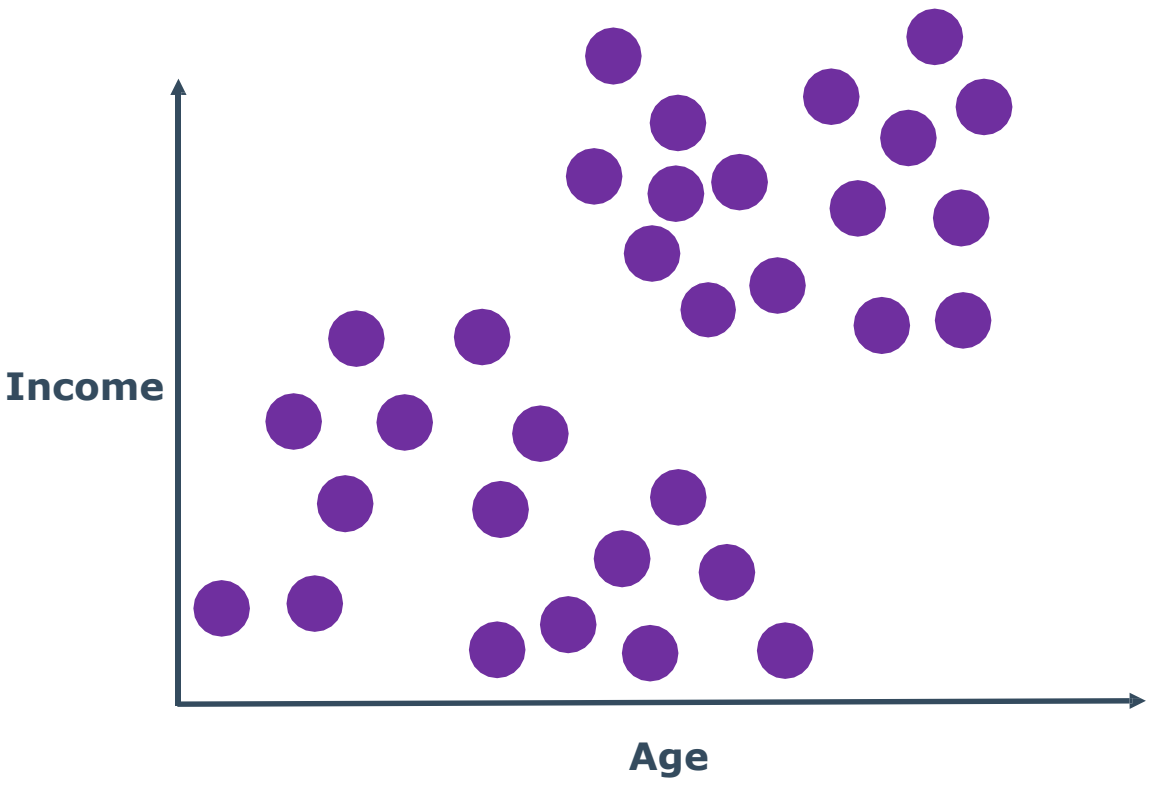
哪个模型正确？

Inertia = 13.112



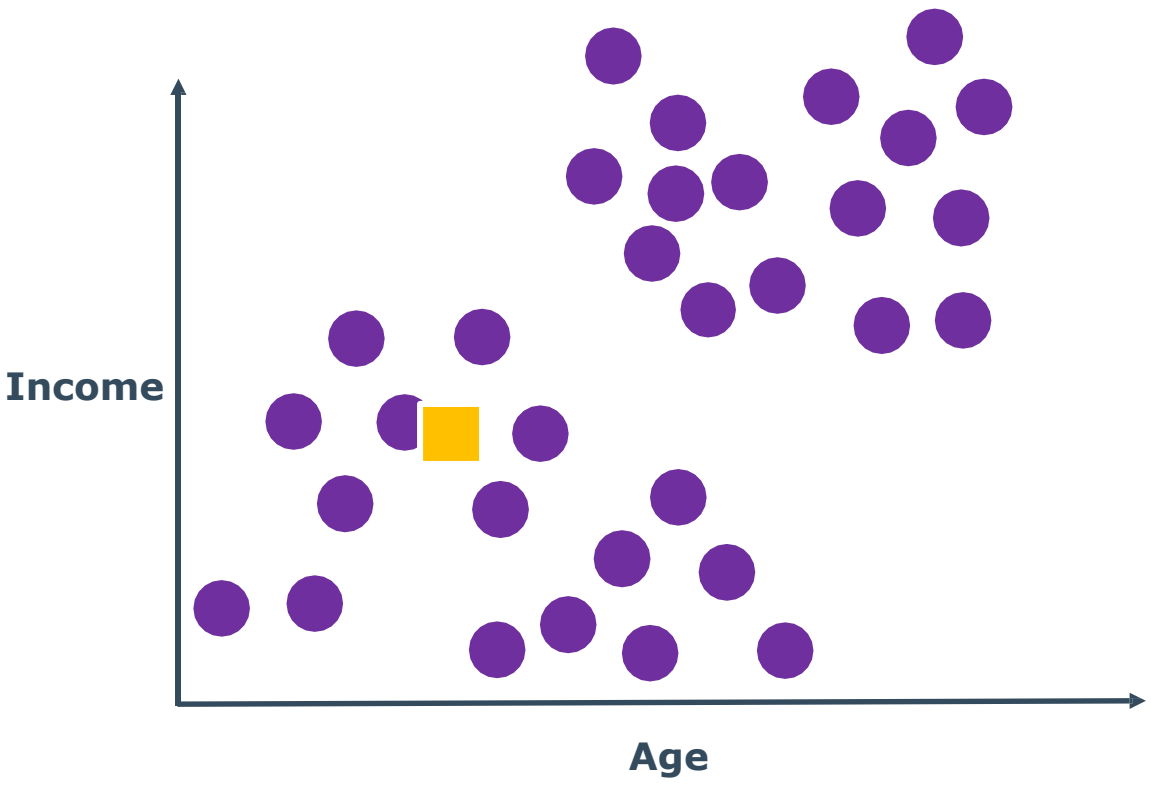
初始聚簇中心的选择方法

更聪明的K-Means初始化方法



更聪明的K-Means初始化方法

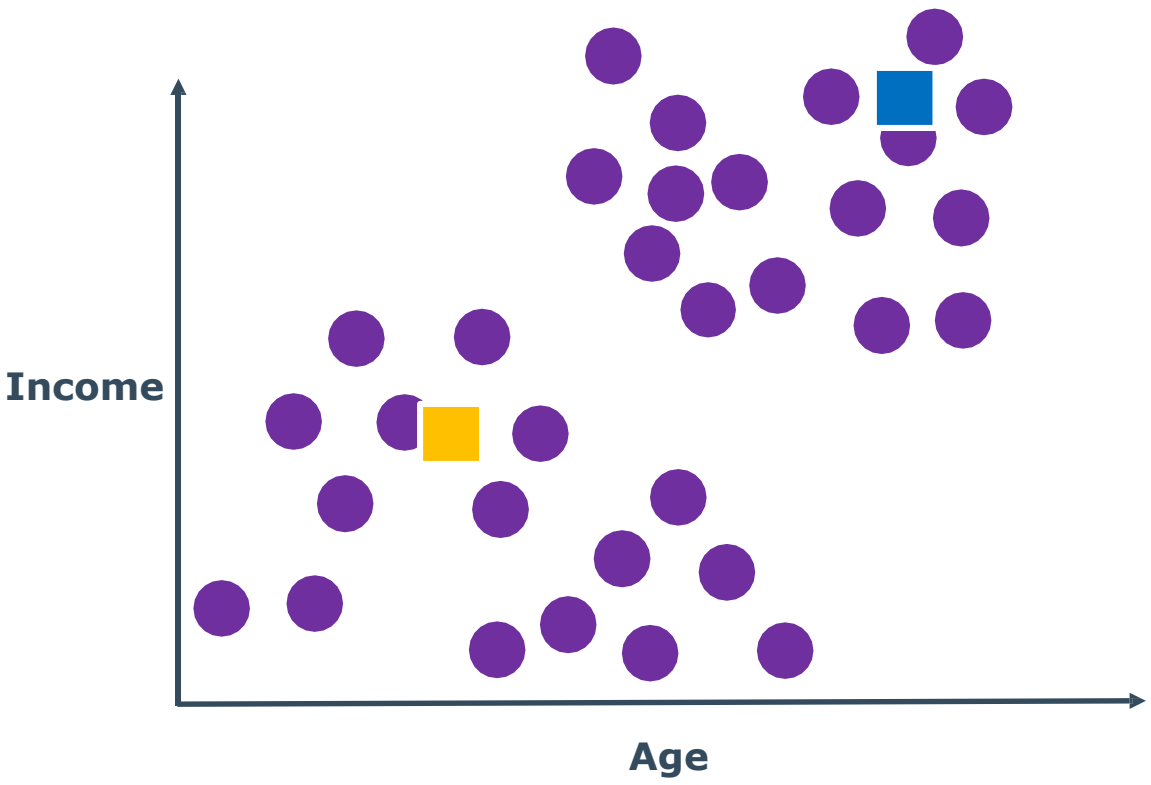
随机地选取一点作为起始点



更聪明的K-Means初始化方法

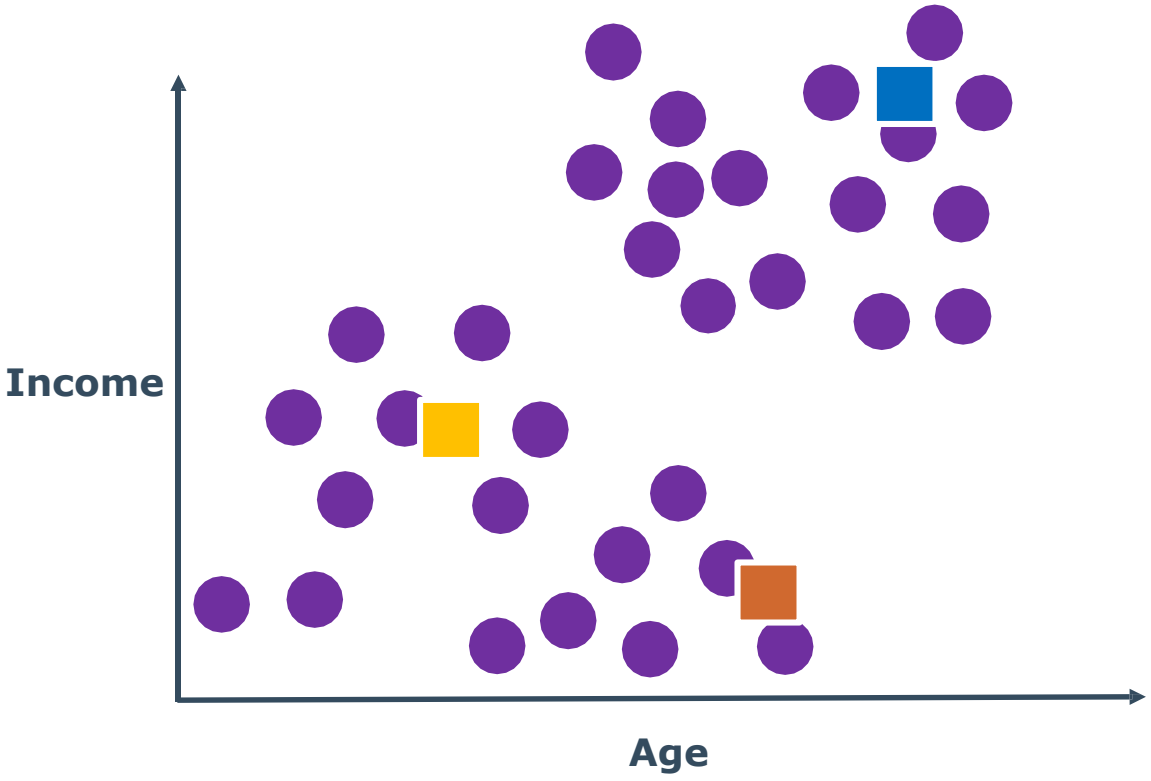
- 计算每个点与已有聚簇中心点之间的最短距离，D(x)
- 按照$D(x)^2/\Sigma D(x)^2$的概率选取下一个点

以概率选择距离最大的样本作为新的聚类中心



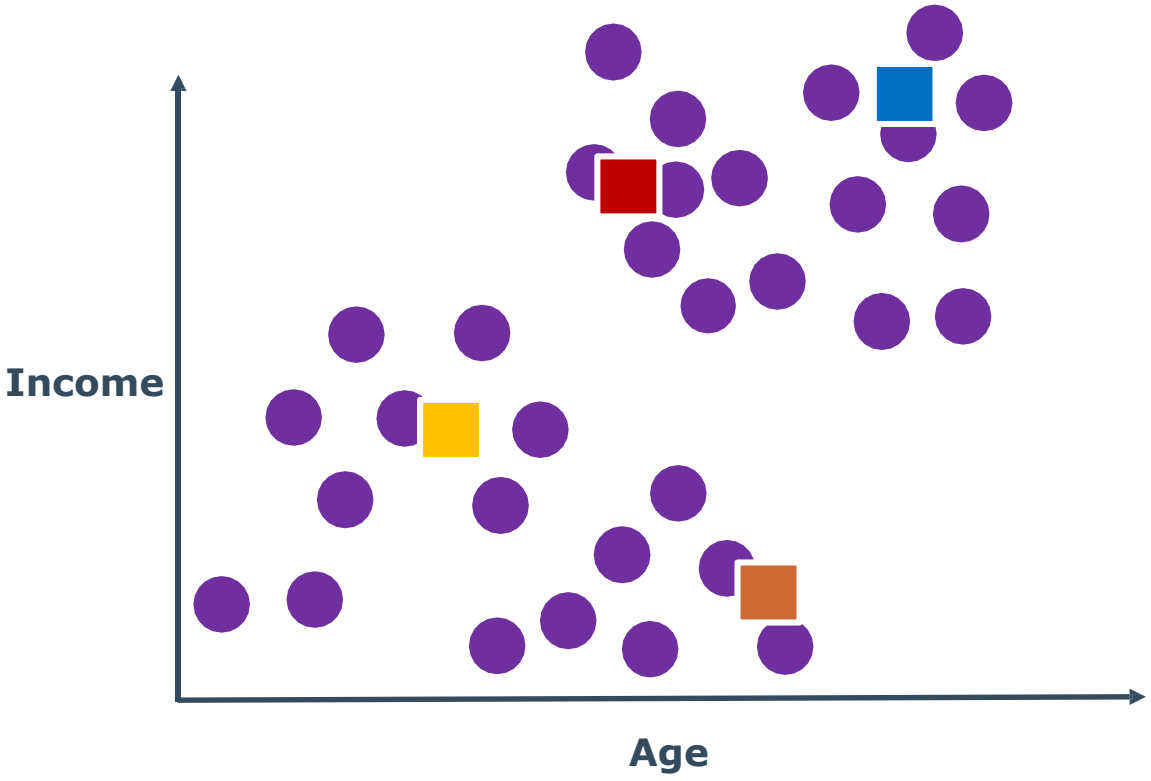
更聪明的K-Means初始化方法

- 计算每个点与已有聚簇中心点之间的最短距离，D(x)
- 按照$D(x)^2/\Sigma D(x)^2$的概率选取下一个点



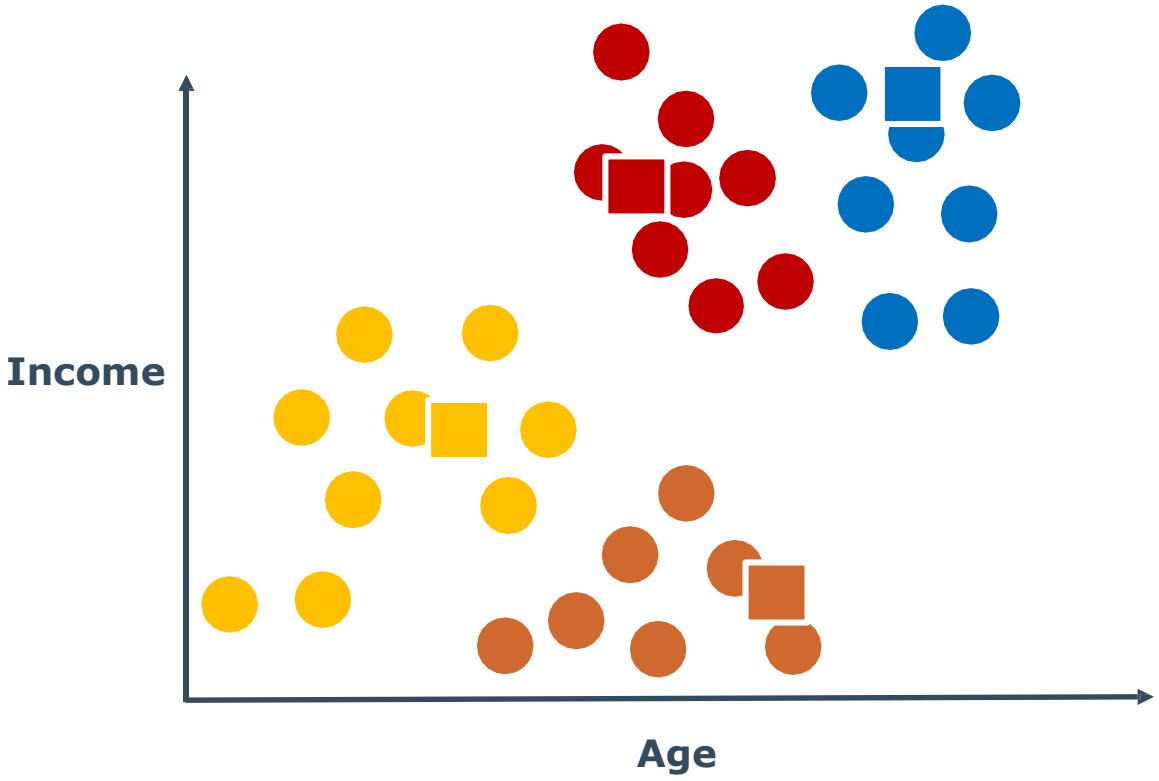
更聪明的K-Means初始化方法

- 计算每个点与已有聚簇中心点之间的最短距离，D(x)
- 按照$D(x)^2/\Sigma D(x)^2$的概率选取下一个点



更聪明的K-Means初始化方法

分配聚簇



K-Means++

K值的选择方法

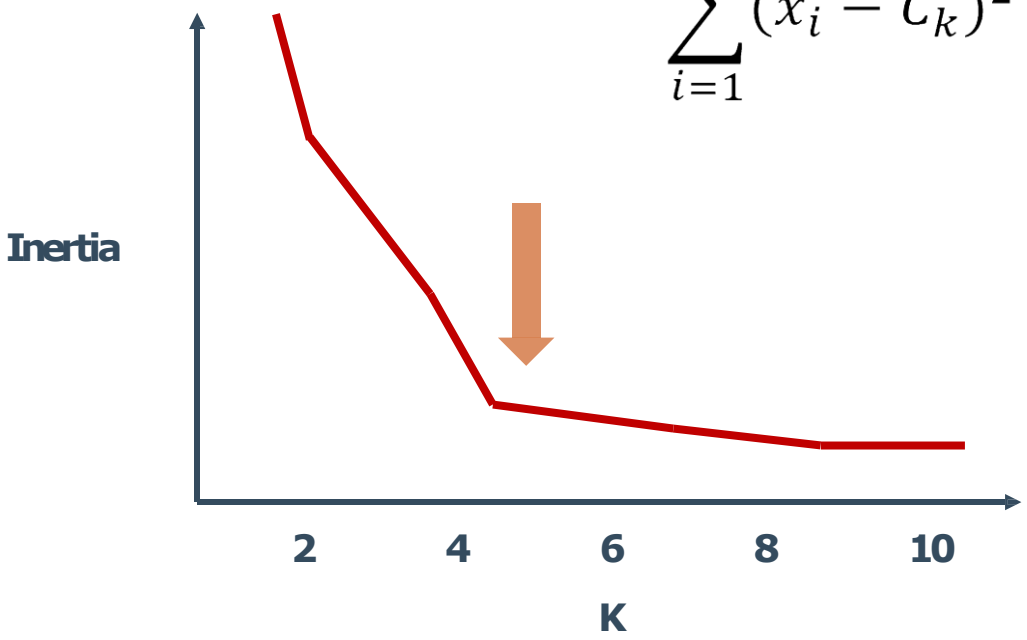
选择正确的聚簇数

- 有时应用中有一个**K**：
  - 将相似的任务聚集在四个**CPU**核上 **(K=4)**
  - 一件服装设计成**10**种不同大小以适合不同的人 **(K=10)**
  - 一个导览界面可以浏览**20**种不同科学领域的论文 **(K=20)**

选择正确的聚簇数

- **Inertia**衡量点到聚簇中心的距离
- 值会随着**K**的增大而不断降低，只要聚簇密度在不断增大
- 小于真实**K**值时，下降幅度很大；超过真实**K**值时，下降趋于平缓

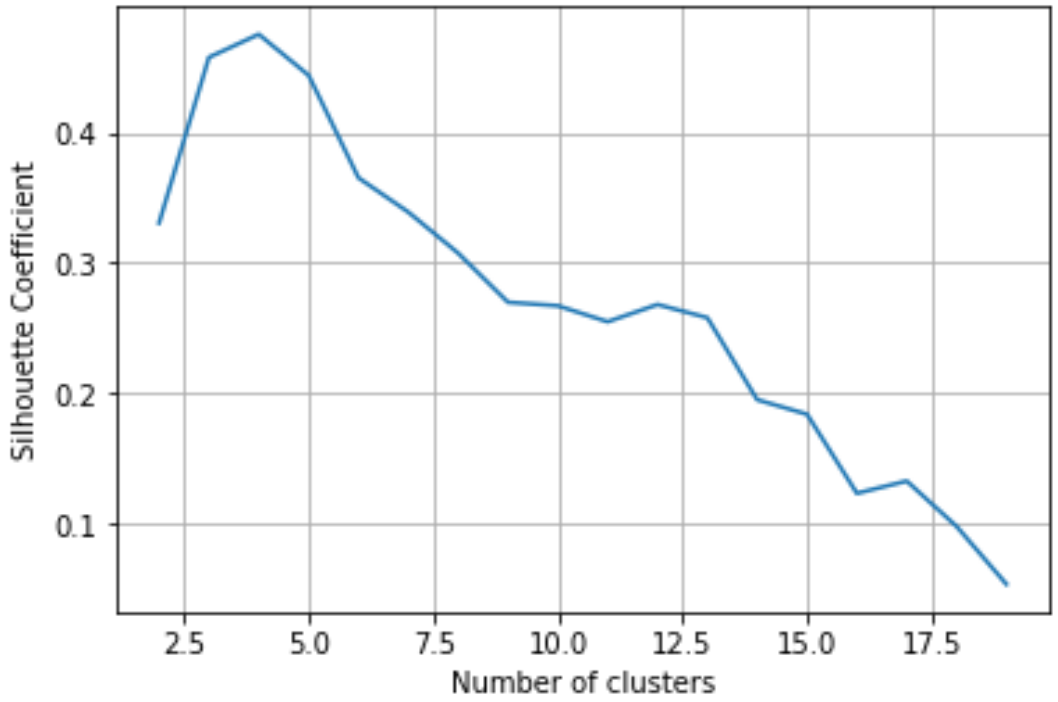
$$\sum_{i=1}^{n}(x_i - C_k)^2$$



手肘法

选择正确的聚簇数

- 选择轮廓系数最大的K值



K-Means的语法

导入包含聚类方法的类：

```
from sklearn.cluster import KMeans
```

创建该类的一个对象：

```
kmeans = KMeans(n_clusters=3,
                init='k-means++')
```

拟合数据，并在新数据上预测聚簇：

```
kmeans = kmeans.fit(X1)
y_predict = kmeans.predict(X2)
```

http://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

K-Means的语法

导入包含聚类方法的类：

```
from sklearn.cluster import KMeans
```

创建该类的一个对象：

```
kmeans = KMeans(n_clusters=3,
                init='k-means++')
```

最终的聚簇数

拟合数据，并在新数据上预测聚簇：

```
kmeans = kmeans.fit(X1)
y_predict = kmeans.predict(X2)
```

K-Means的语法

导入包含聚类方法的类：

```
from sklearn.cluster import KMeans
```

创建该类的一个对象：

```
kmeans = KMeans(n_clusters=3,
                init='k-means++')
```

kmeans++
初始化方法

拟合数据，并在新数据上预测聚簇：

```
kmeans = kmeans.fit(X1)
y_predict = kmeans.predict(X2)
```

K-Means的语法

导入包含聚类方法的类：

```
from sklearn.cluster import KMeans
```

创建该类的一个对象：

```
kmeans = KMeans(n_clusters=3,
                init='k-means++')
```

拟合数据，并在新数据上预测聚簇：

```
kmeans = kmeans.fit(X1)
y_predict = kmeans.predict(X2)
```

也可以用**MiniBatchKMeans**使用批处理方式

13.1 什么是聚类

13.2 K-MEANS算法

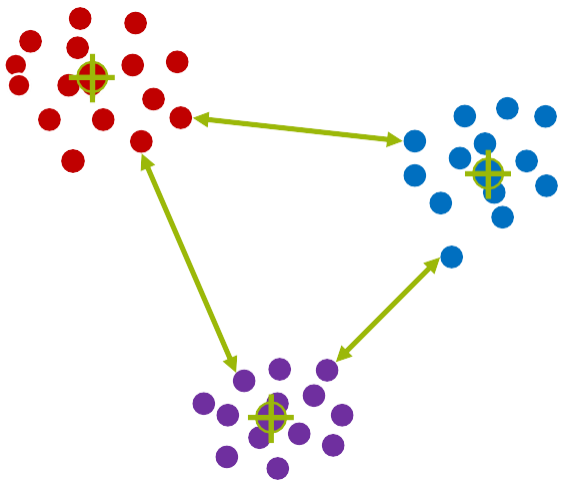
13.3 聚类结果的评价

13.4 不同的距离指标

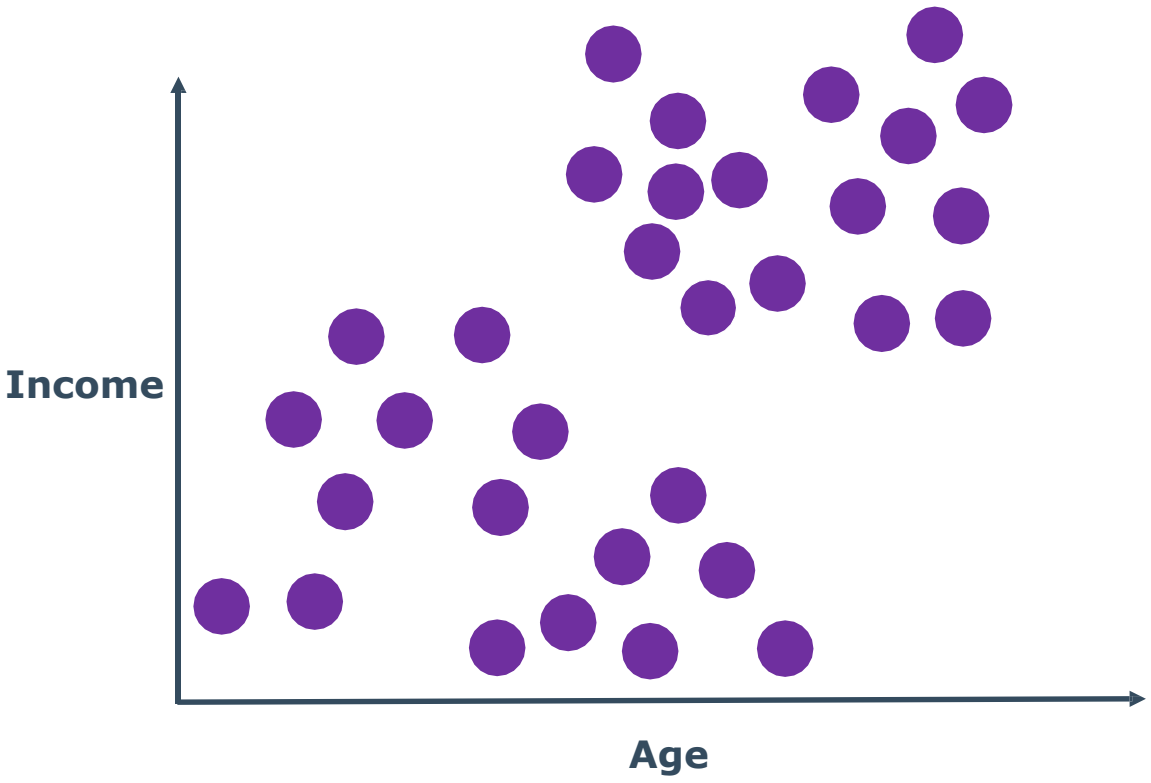
13.5 聚合式层次聚类

距离指标的选择

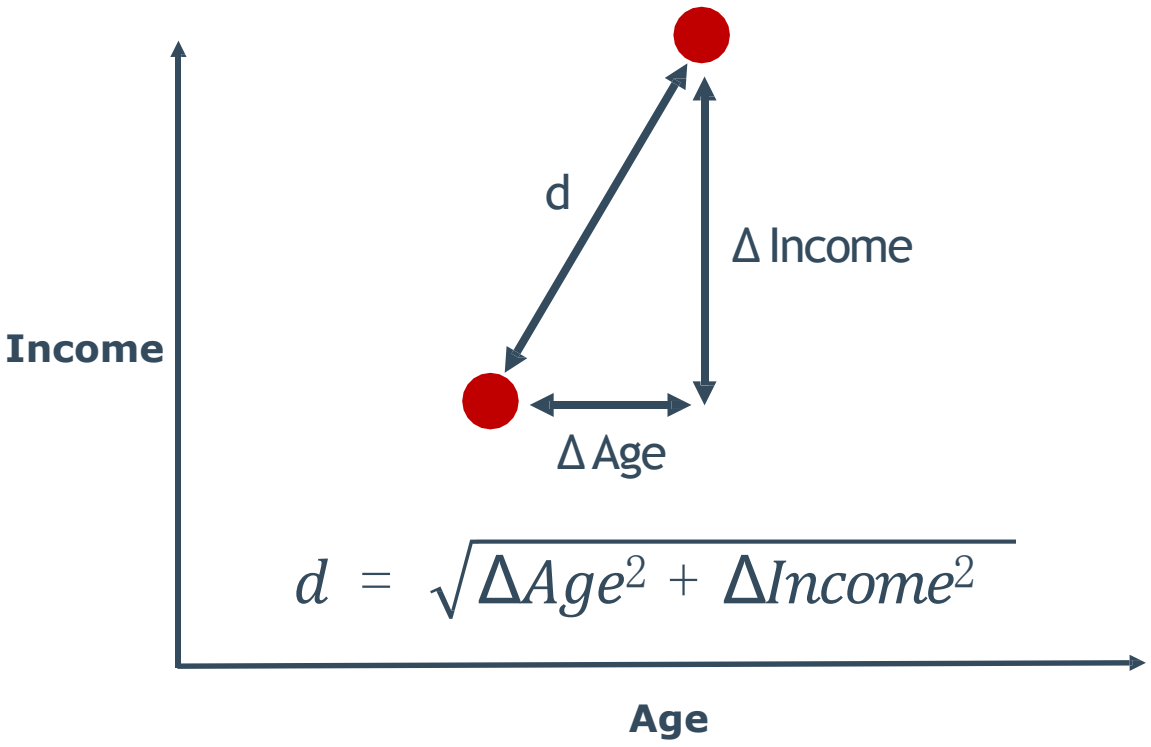
- 距离指标的选择对聚类的成功至关重要
- 每个指标有各自的优点和适用情况
- 但有时距离指标的选择也是基于经验性的评价



欧几里得距离

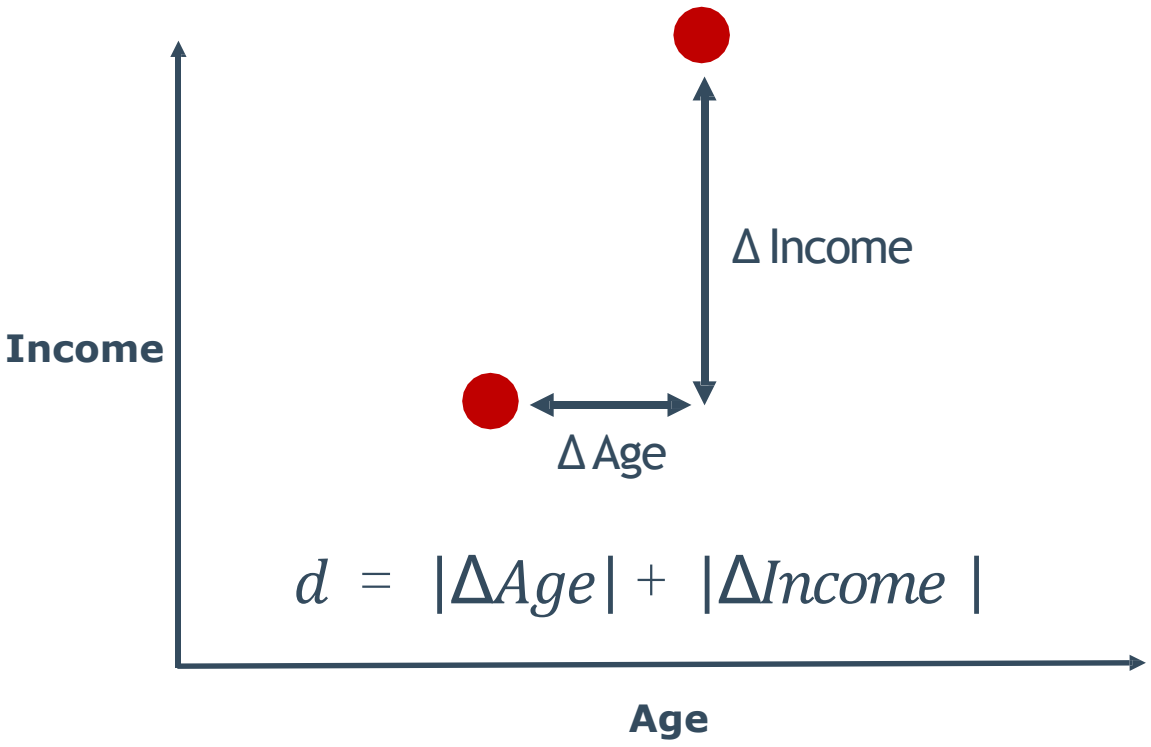


欧几里得距离（L2距离）



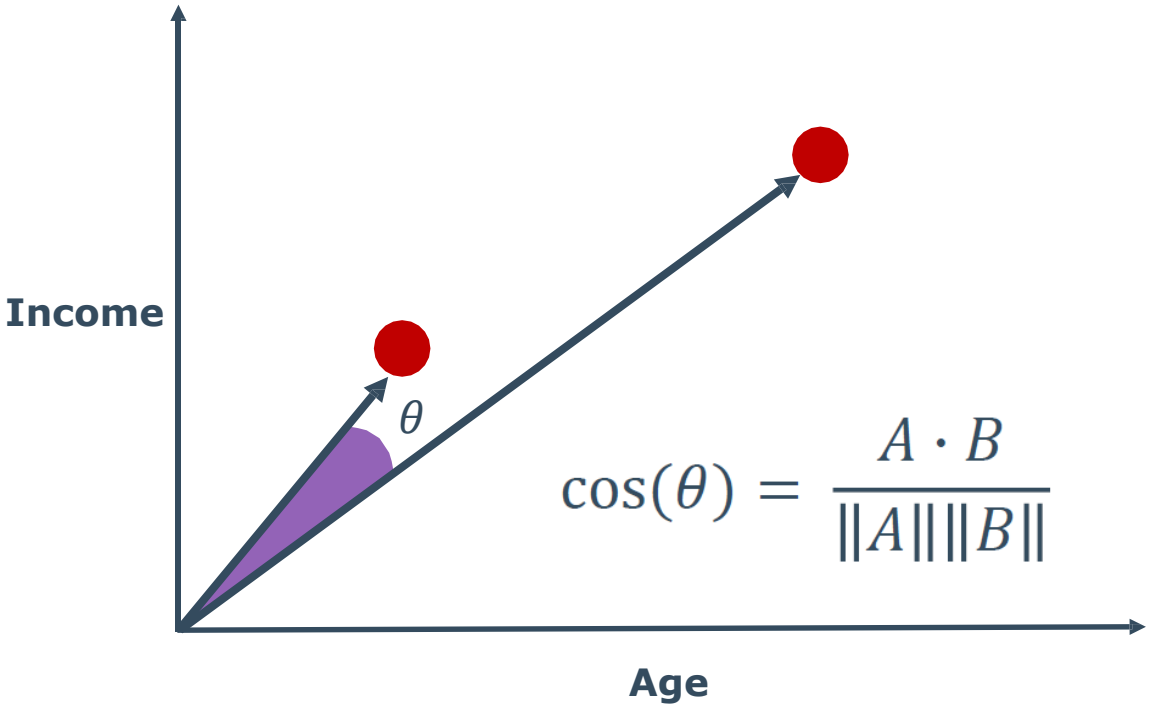
$$d = \sqrt{\Delta Age^2 + \Delta Income^2}$$

曼哈顿距离（L1或街区距离）

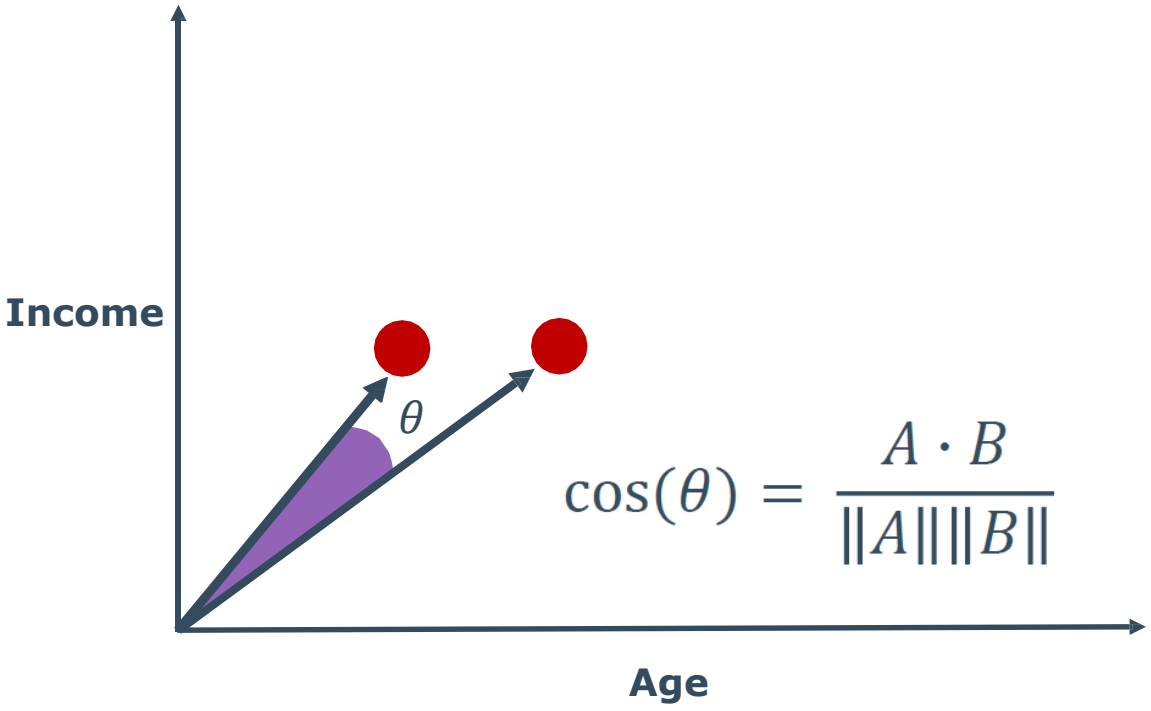


$$d = |\Delta Age| + |\Delta Income|$$

余弦距离



余弦距离



$$\cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

欧几里得vs. 余弦距离

- 欧几里得距离适用于基于坐标的度量
- 余弦距离更适合那些出现位置不重要的数据，例如文本数据
- 欧几里得距离对维度灾难更敏感

Jaccard距离

应用于集合（例如单词的出现）

- 句子 **A**: “I like chocolate ice cream.”
- set A = {I, like, chocolate, ice, cream}
- 句子 **B**: “Do I want chocolate cream or vanilla cream?”
- set B = {Do, I, want, chocolate, cream, or, vanilla}

$$1 - \frac{A \cap B}{A \cup B} = 1 - \frac{len(shared)}{len(unique)}$$

Jaccard距离

应用于集合（例如单词的出现）

- 句子 **A**: “I like chocolate ice cream.”
- set A = {**I**, like, **chocolate**, ice, **cream**}
- 句子 **B**: “Do I want chocolate cream or vanilla cream?”
- set B = {Do, **I**, want, **chocolate**, **cream**, or, vanilla}

$$1 - \frac{A \cap B}{A \cup B} = 1 - \frac{3}{9}$$

距离指标的语法

导入一般的两两距离计算函数：

```
from sklearn.metrics import pairwise_distances
```

计算距离：

```
dist = pairwise_distances(X,Y,
                          metric='euclidean')
```

选择的距离指标

距离指标的语法

导入一般的两两距离计算函数：

```
from sklearn.metrics import pairwise_distances
```

计算距离：

```
dist = pairwise_distances(X,Y,
                          metric='euclidean')
```

其他的距离指标选择有：cosine, manhattan, jaccard, 等等

距离指标的语法

导入一般的两两距离计算函数：

```
from sklearn.metrics import pairwise_distances
```

计算距离：

```
dist = pairwise_distances(X,Y,
                          metric='euclidean')
```

其他的距离指标选择有：cosine, manhattan, jaccard, 等等

距离指标函数也可以被专门导入，如：

```
from sklearn.metrics import euclidean_distances
```

13.1 什么是聚类

13.2 K-MEANS算法

13.3 聚类结果的评价

13.4 不同的距离指标

13.5 聚合式层次聚类

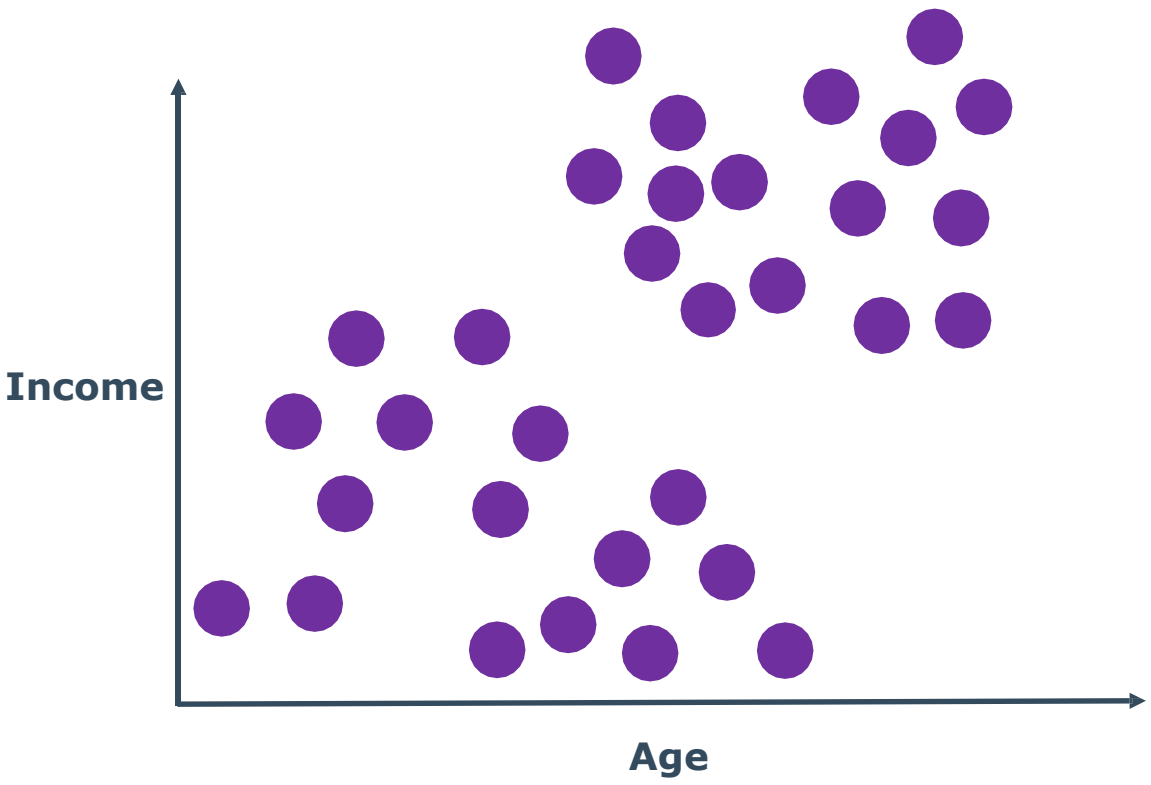
层次聚类

- 层次聚类试图在不同层次对数据集进行划分，从而形成树形的聚类结构。数据集划分既可采用“自底向上”的聚合策略，也可采用“自顶向下”的分拆策略。
- AGNES算法（自底向上的层次聚类算法）

首先，将样本中的每一个样本看做一个初始聚类簇，然后在算法运行的每一步中找出距离最近的两个聚类簇进行合并，该过程不断重复，直到达到预设的聚类簇的个数。

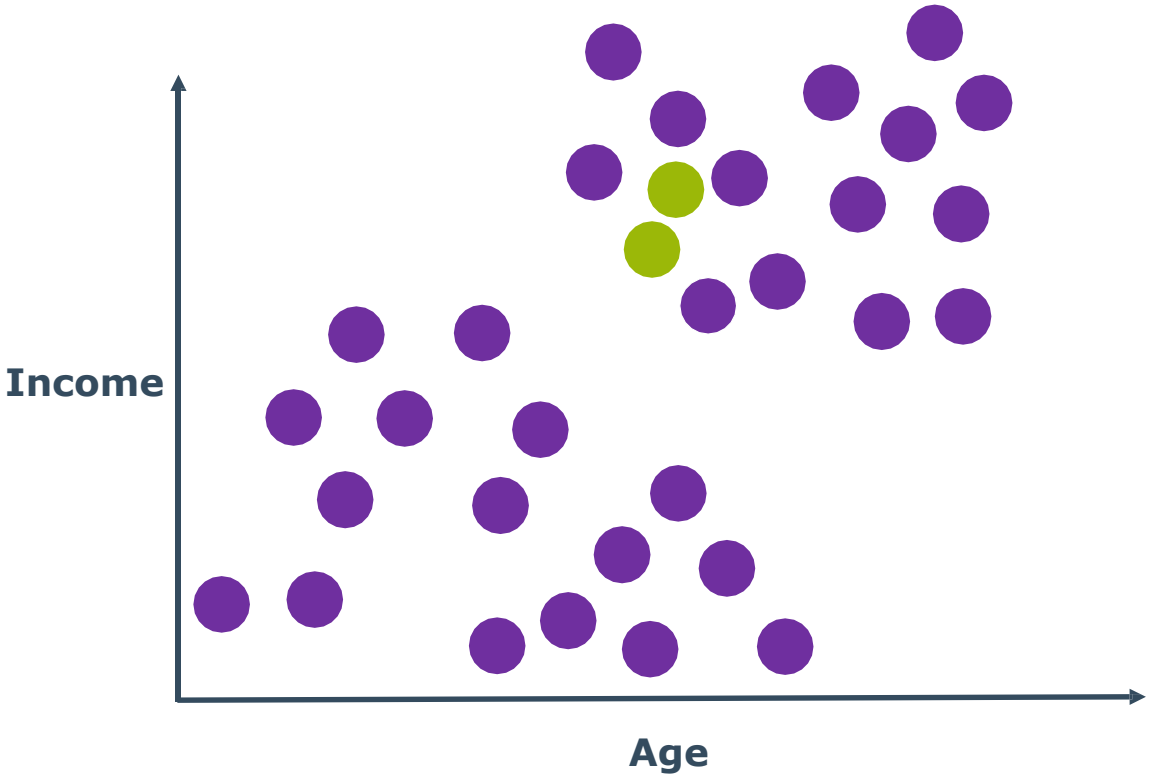
这里两个聚类簇 C_i 和C_j 的距离，可以有多种度量方式。

聚合式层次聚类



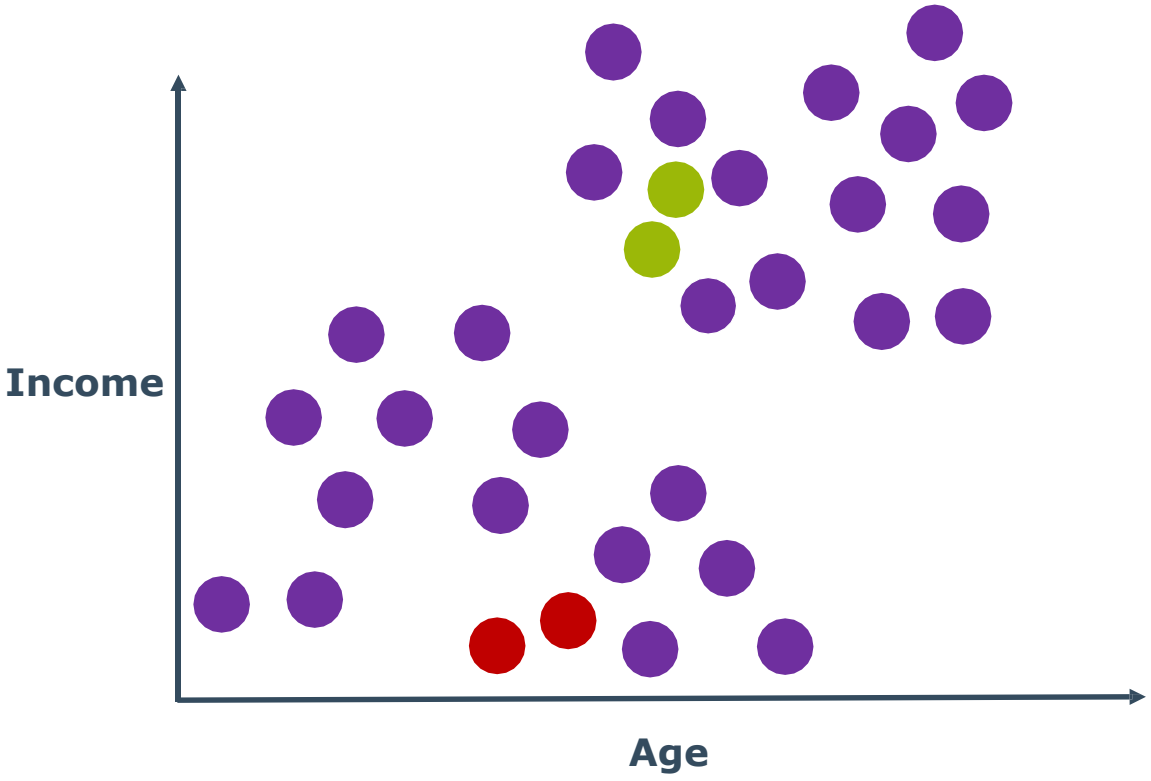
聚合式层次聚类

寻找最近的一对点，
聚成一个聚簇



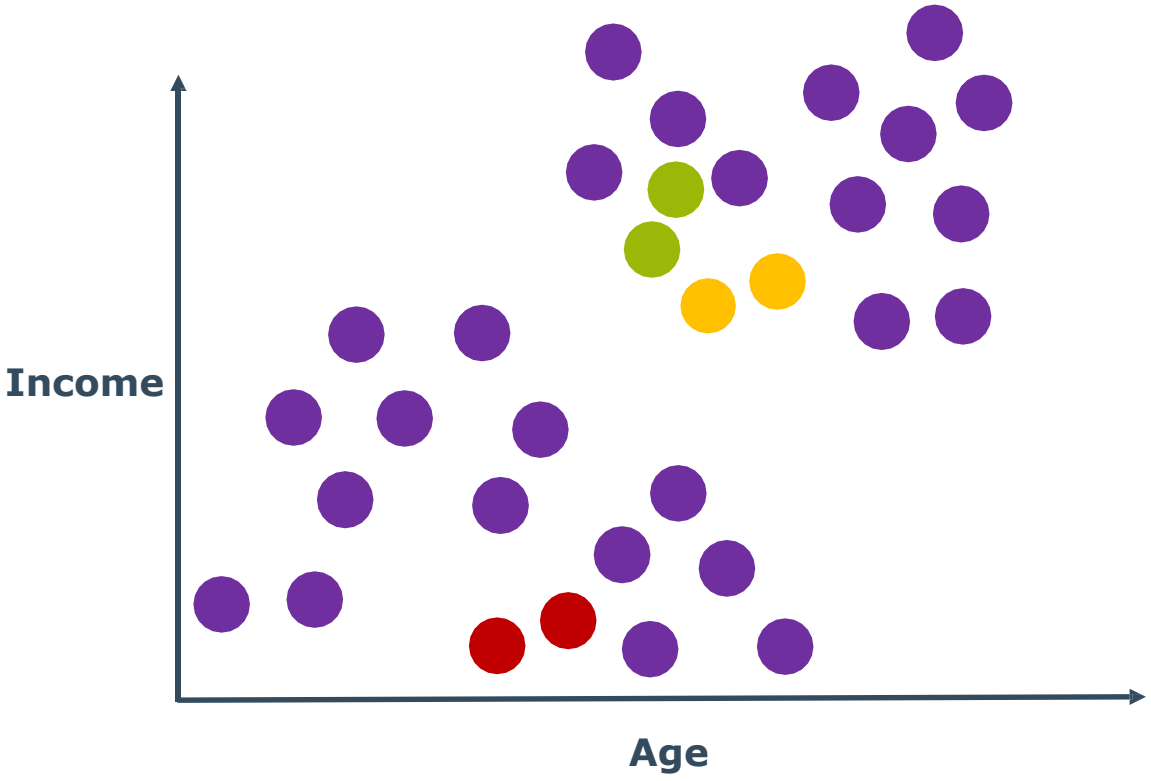
聚合式层次聚类

寻找下一对最近的点，并合并



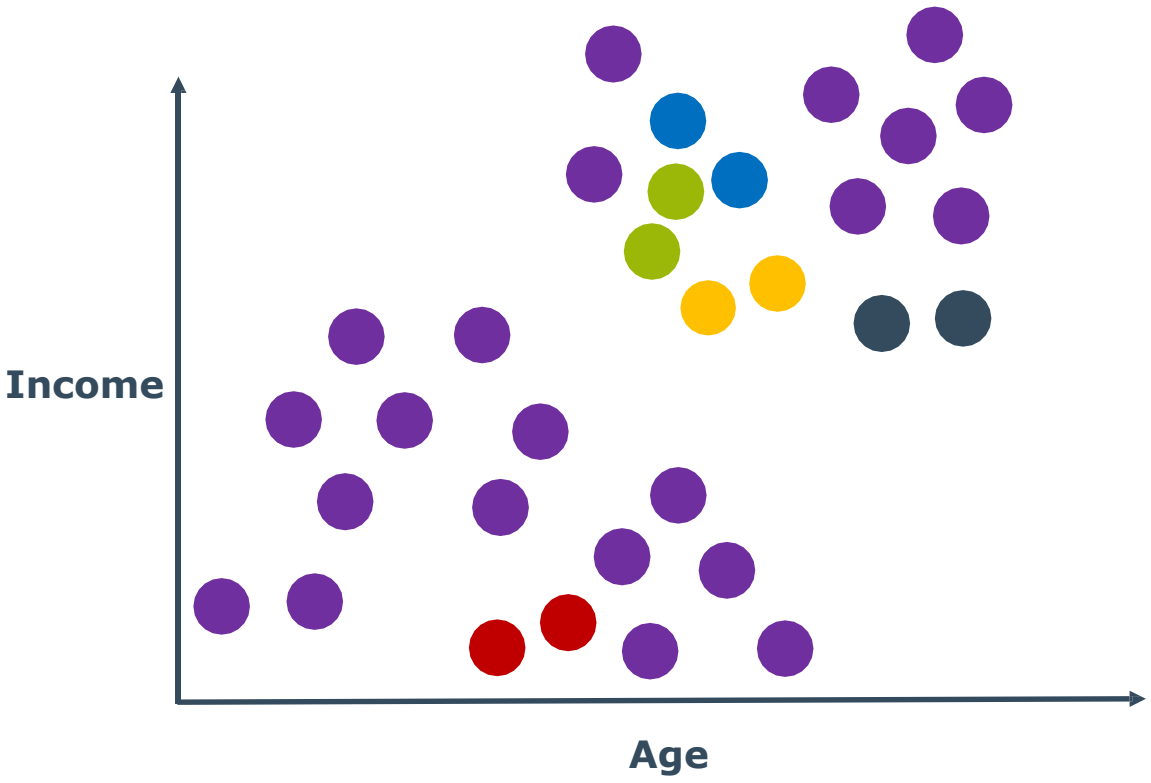
聚合式层次聚类

寻找下一对最近的点，并合并



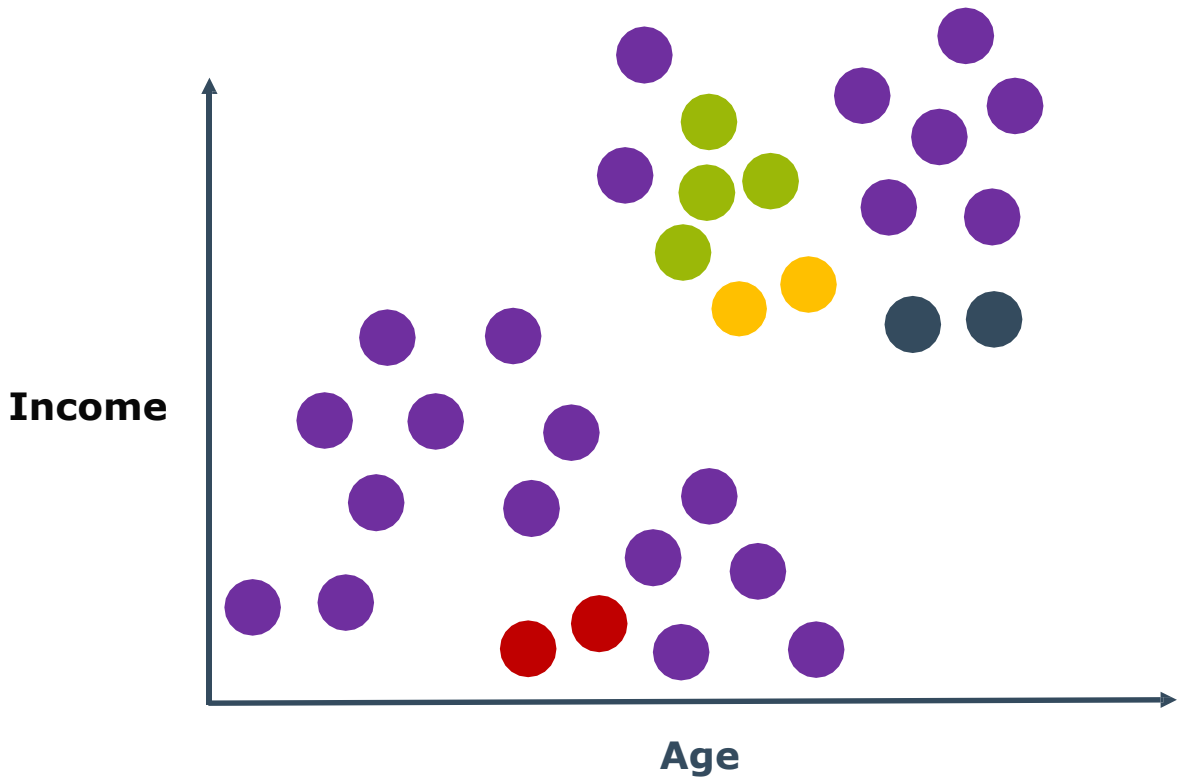
聚合式层次聚类

继续合并距离最近的点



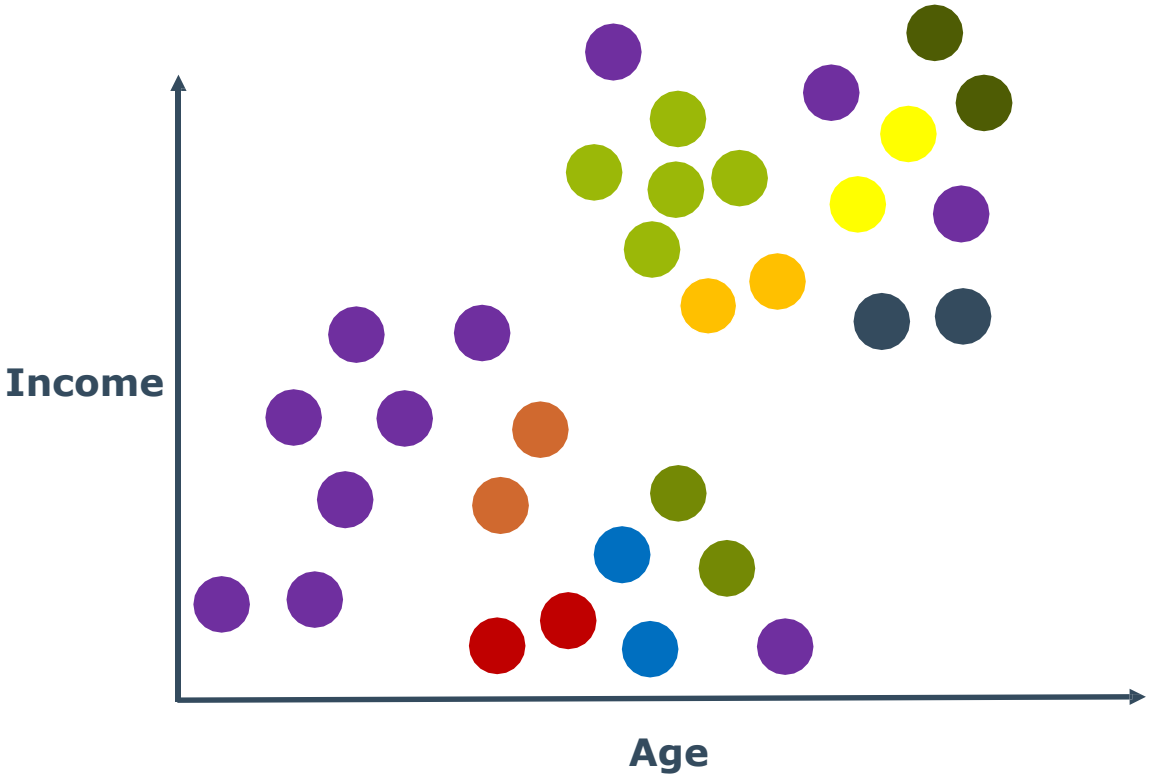
聚合式层次聚类

如果距离最近的是两个聚簇，则合并它们



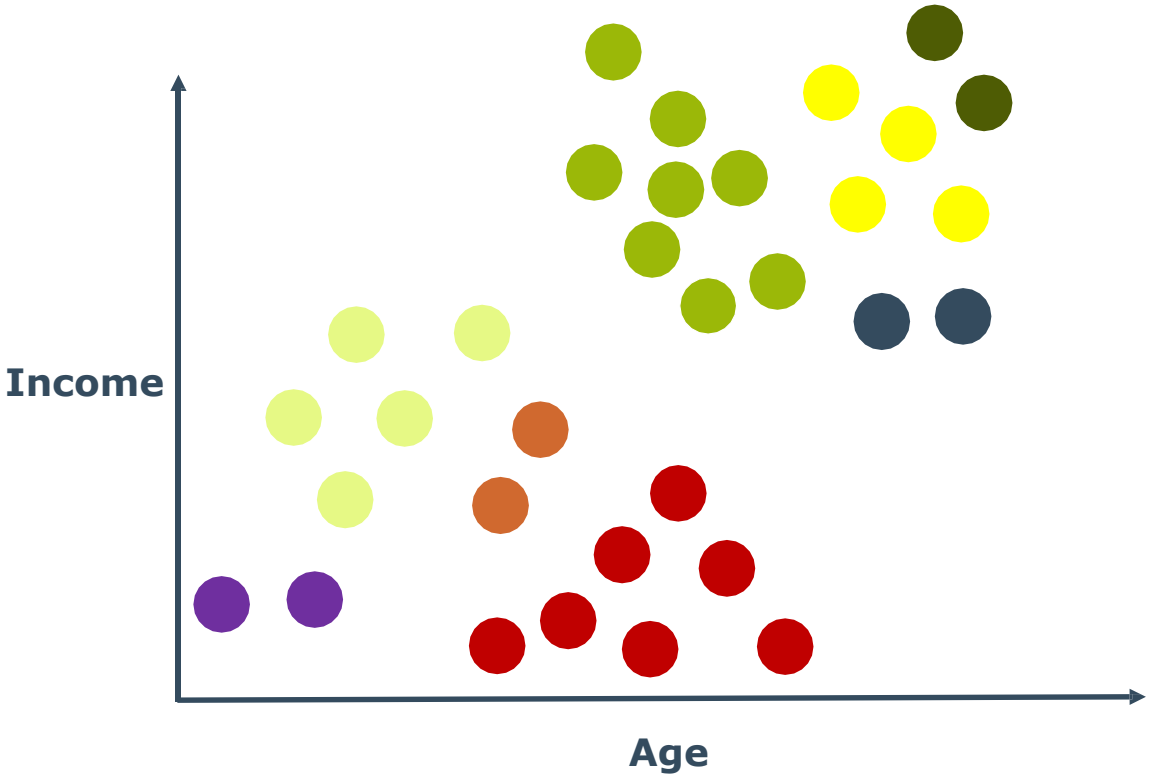
聚合式层次聚类

继续合并距离最近的点和聚簇



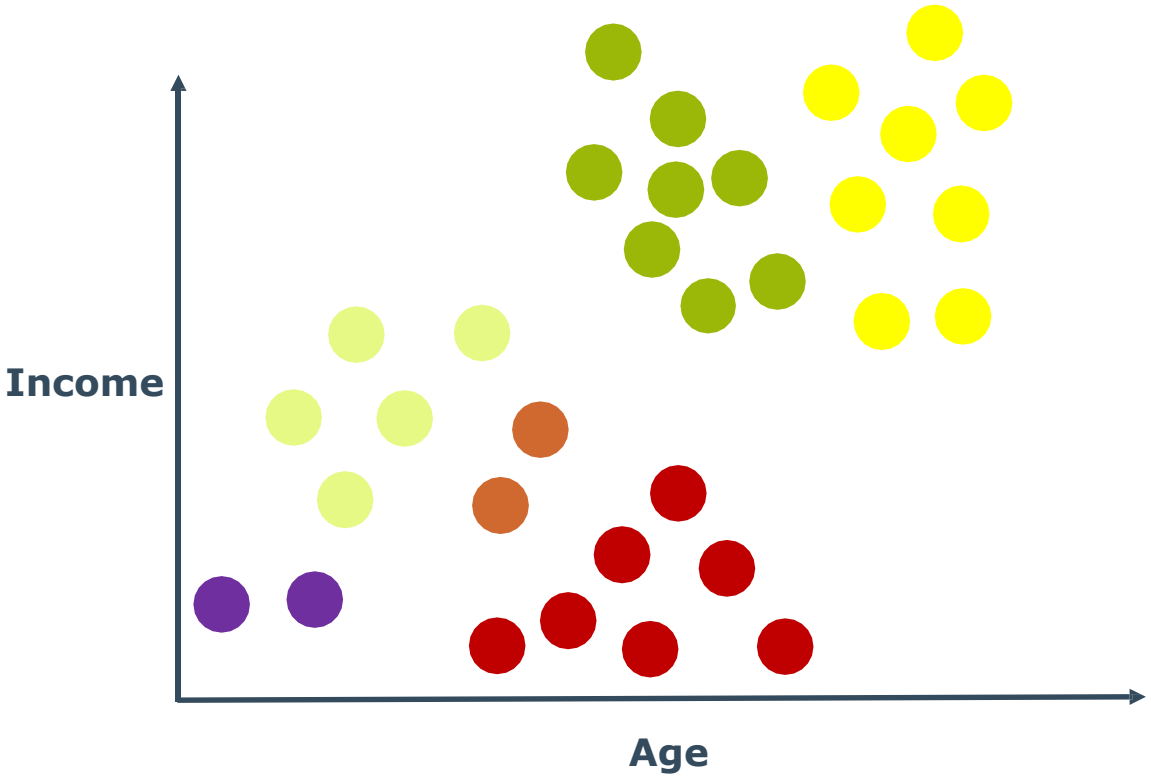
聚合式层次聚类

继续合并距离最近的点和聚簇



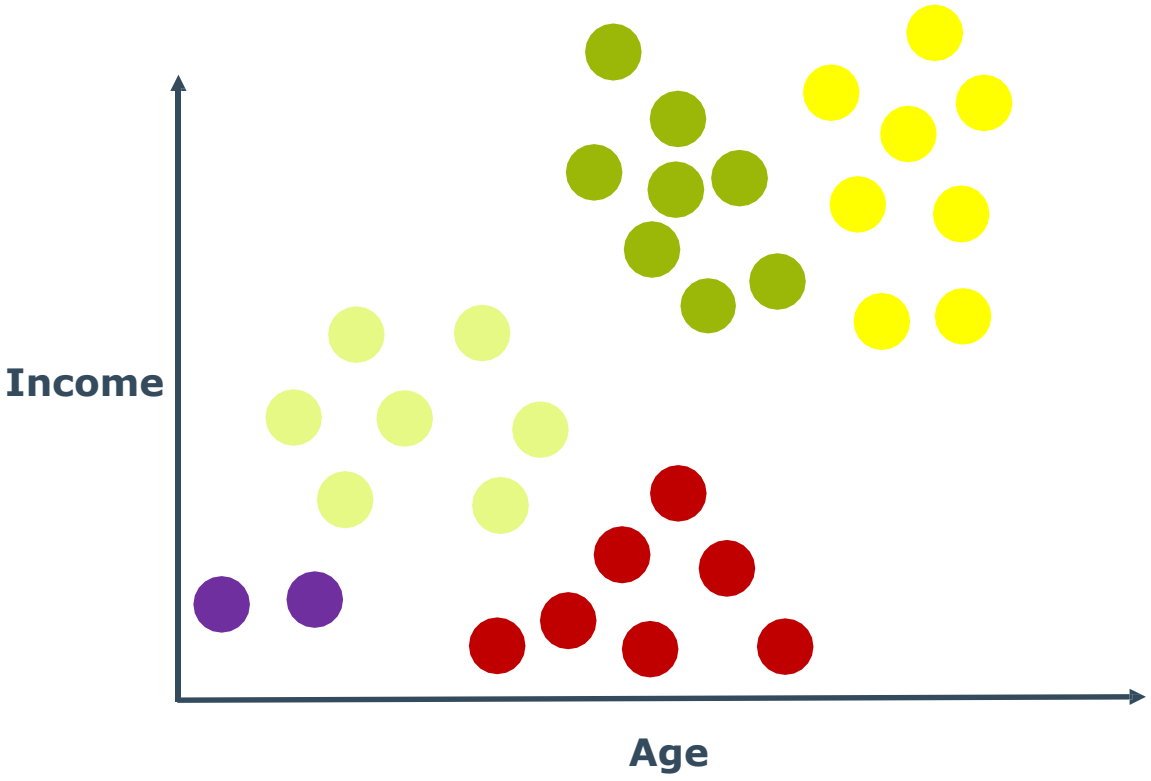
聚合式层次聚类

当前聚簇数 = 6



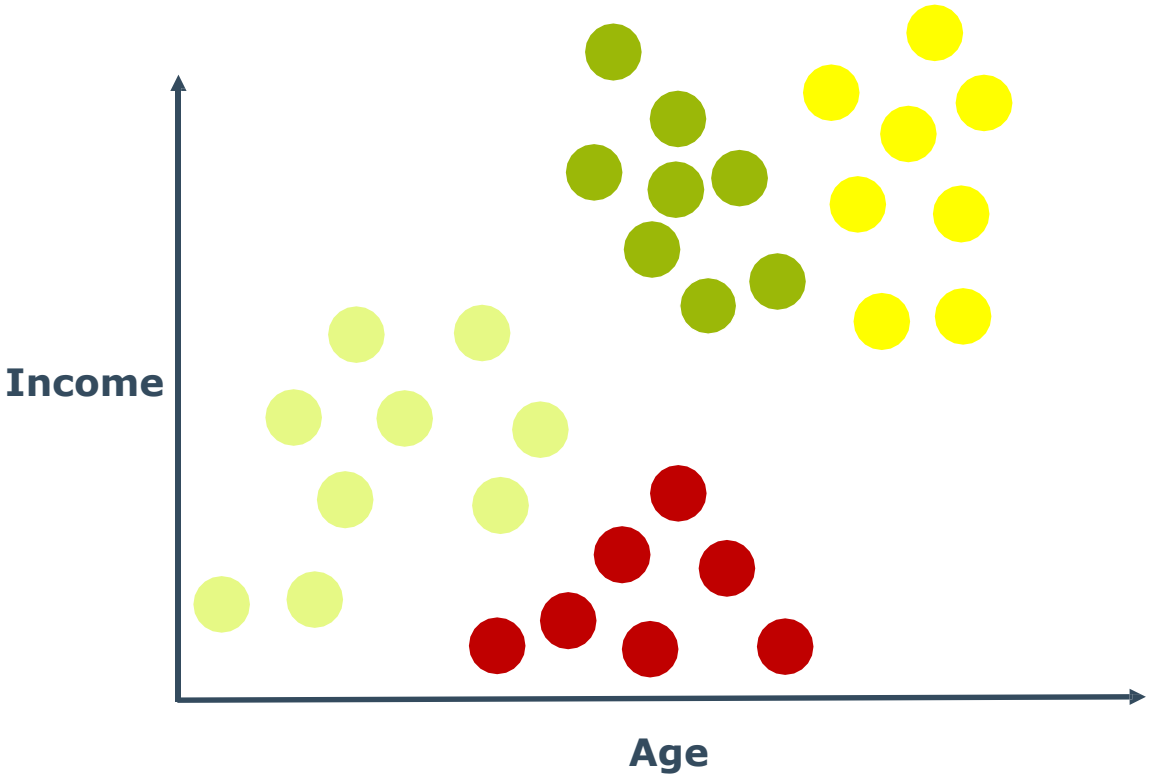
聚合式层次聚类

当前聚簇数 = 5



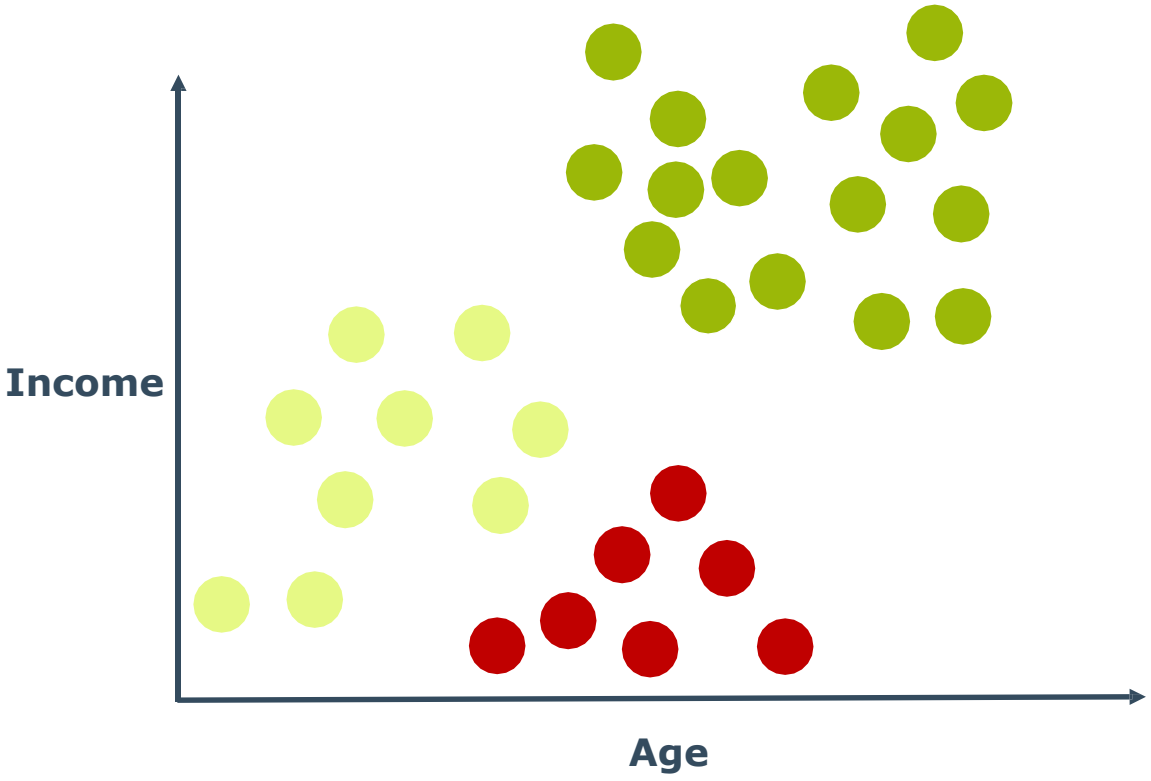
聚合式层次聚类

当前聚簇数 = 4



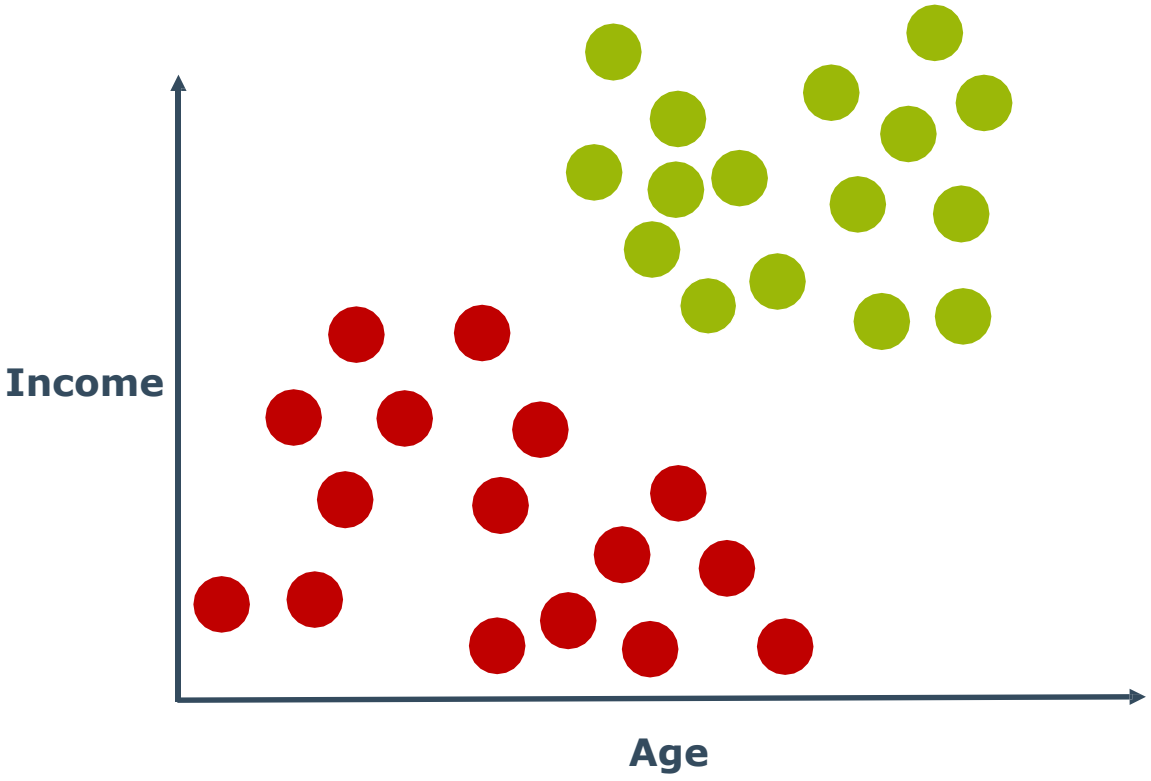
聚合式层次聚类

当前聚簇数 = 3



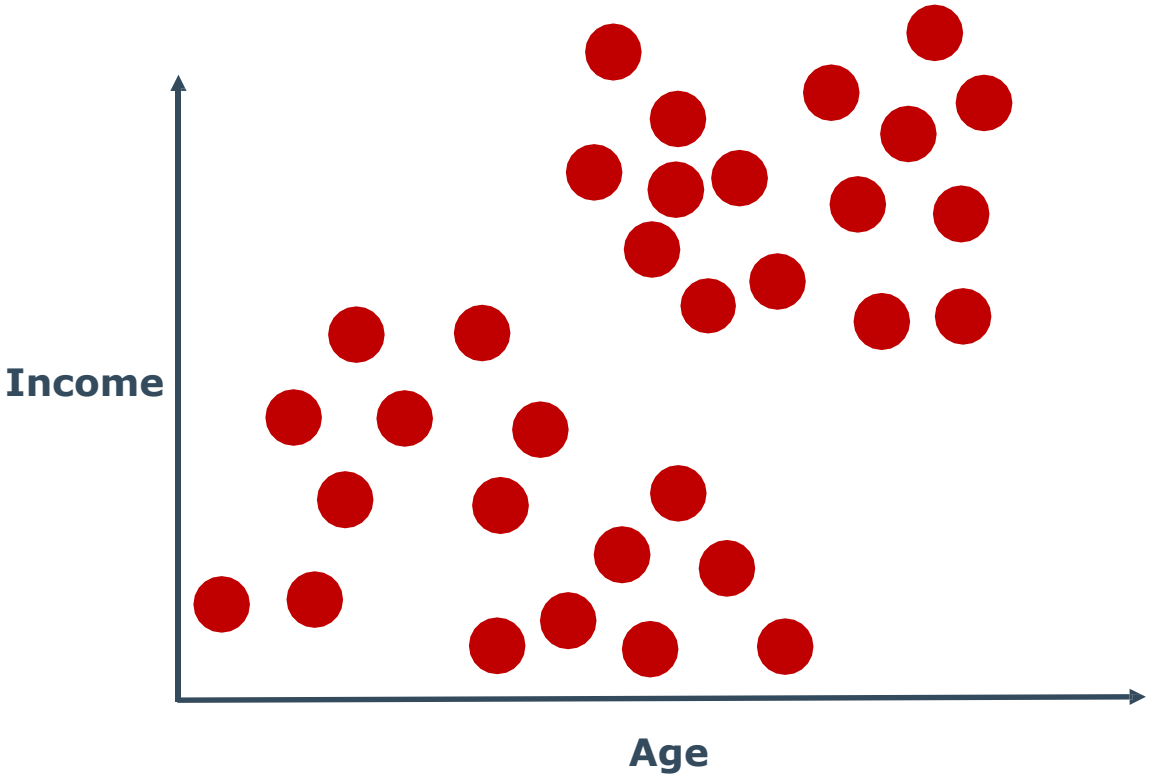
聚合式层次聚类

当前聚簇数 = 2



聚合式层次聚类

当前聚簇数 = 1

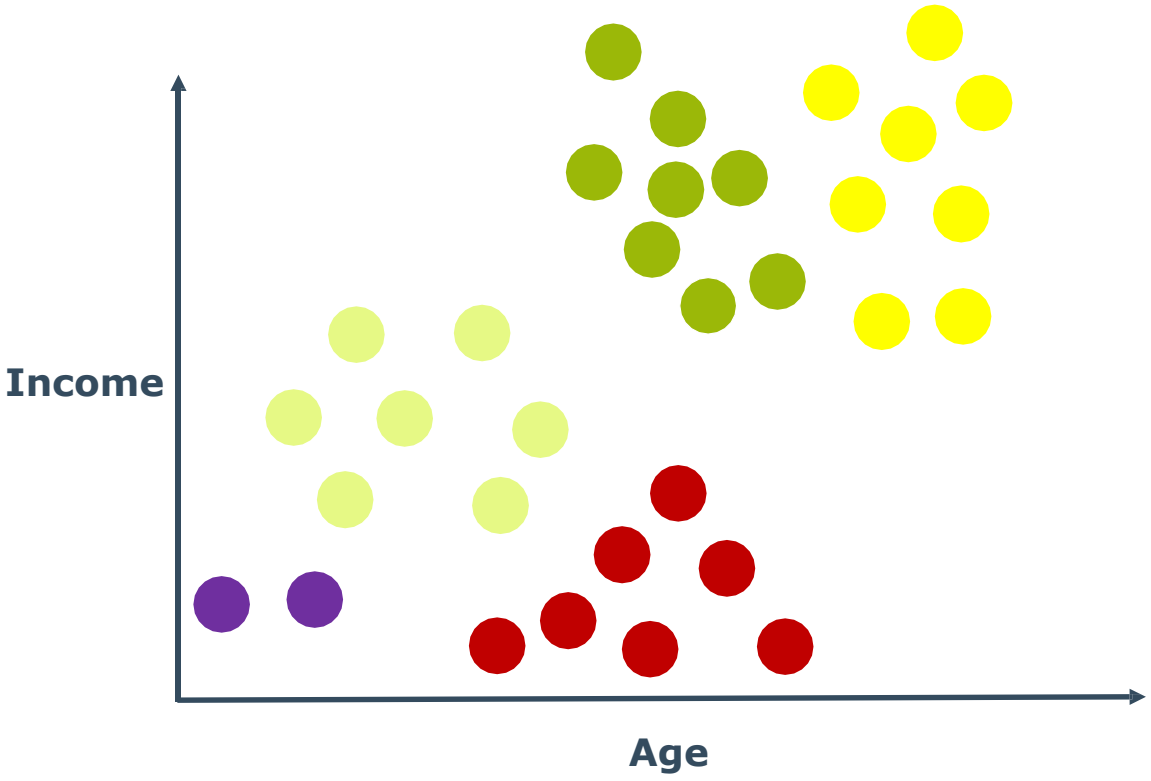


聚合式层次聚类的停止条件

条件 1	达到正确的聚类数
条件 2	最小平均聚簇距离达到预设的值

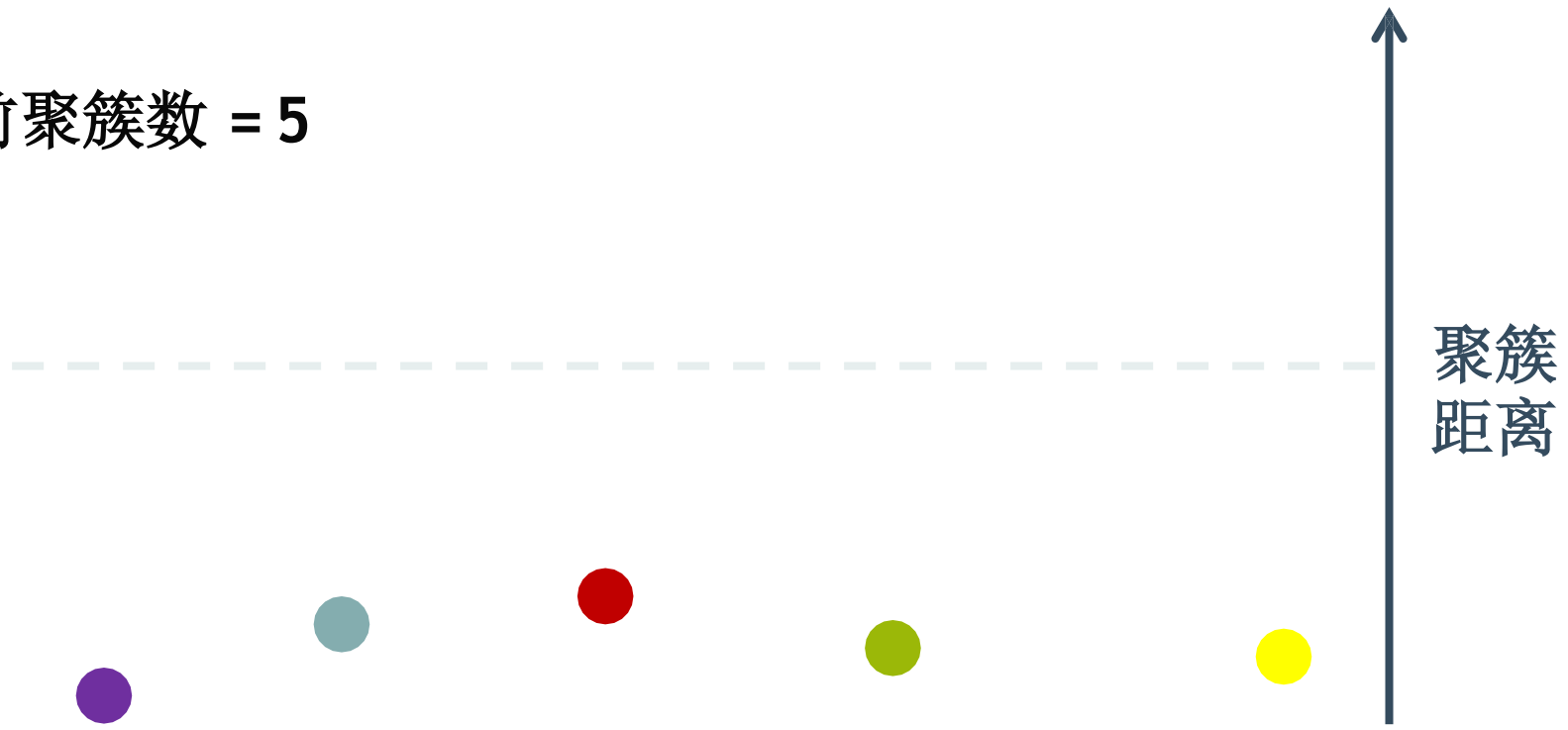
聚合式层次聚类

当前聚簇数 = 5



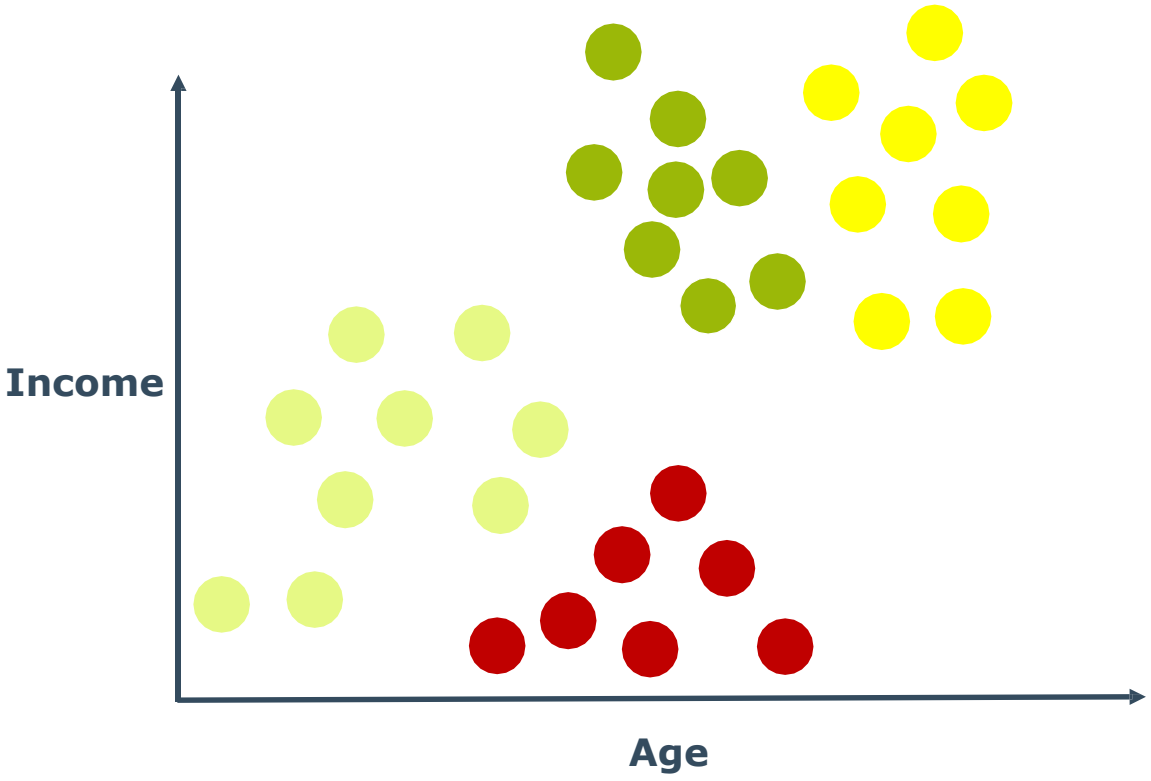
聚合式层次聚类

当前聚簇数 = 5



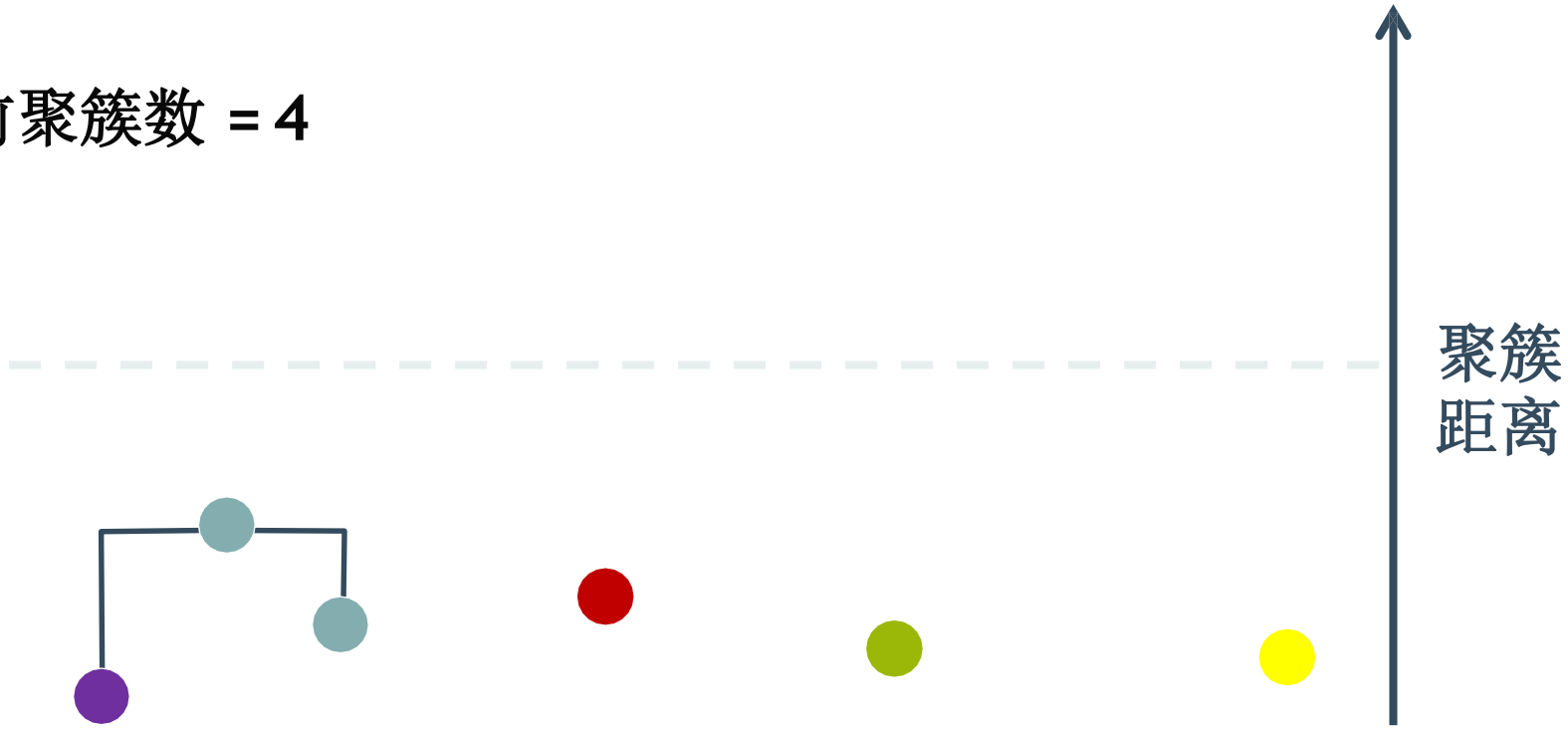
聚合式层次聚类

当前聚簇数 = 4



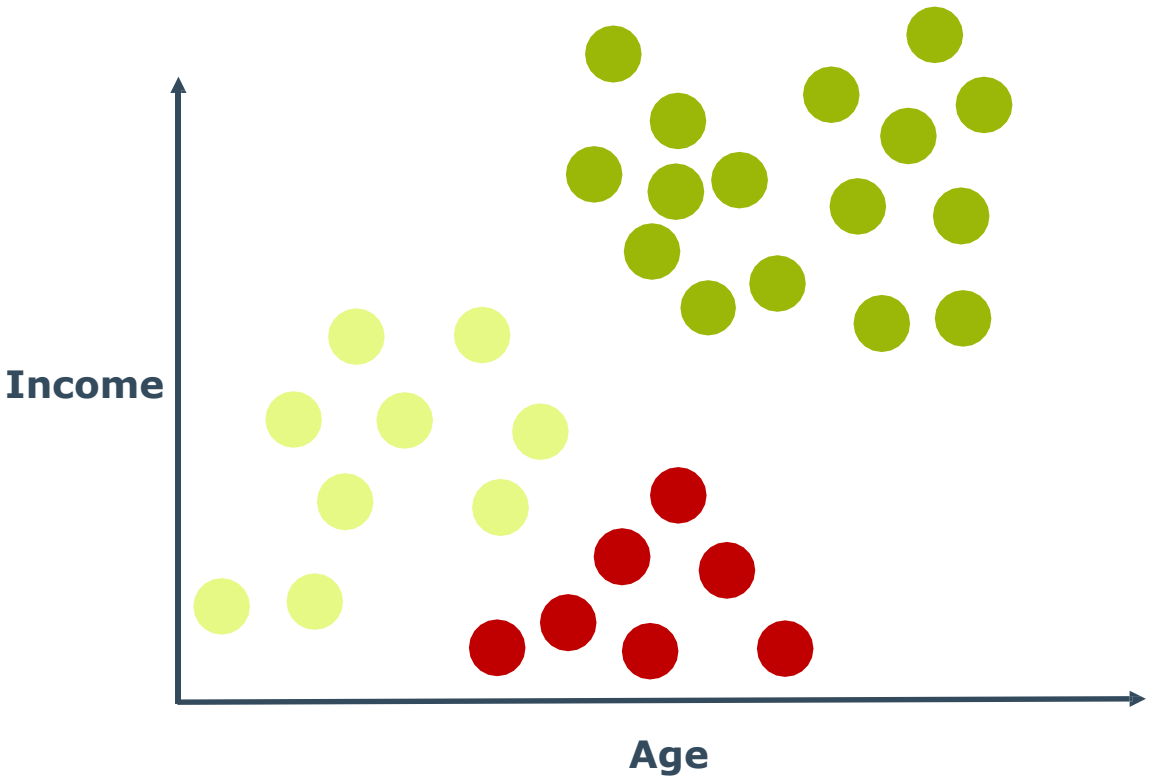
聚合式层次聚类

当前聚簇数 = 4



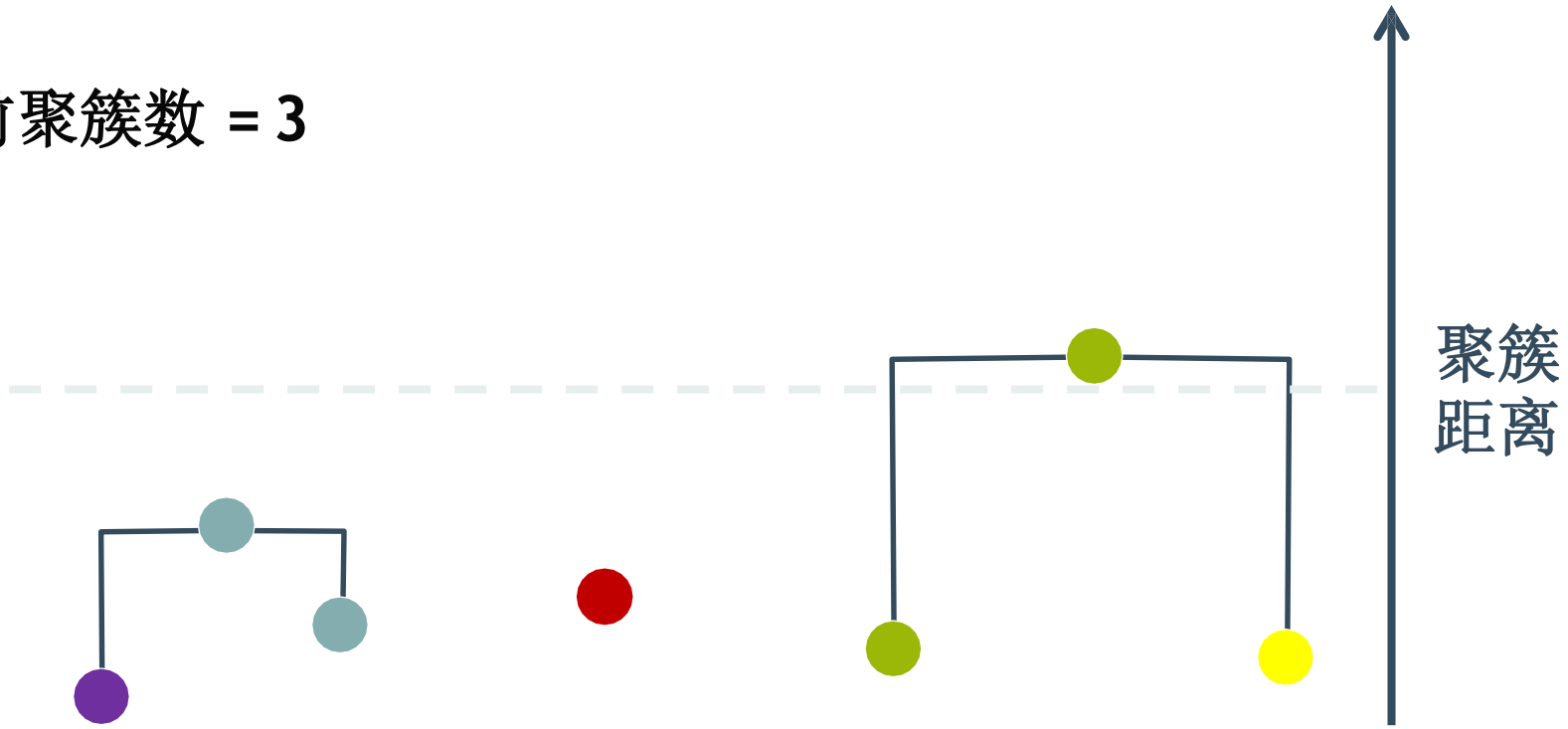
聚合式层次聚类

当前聚簇数 = 3



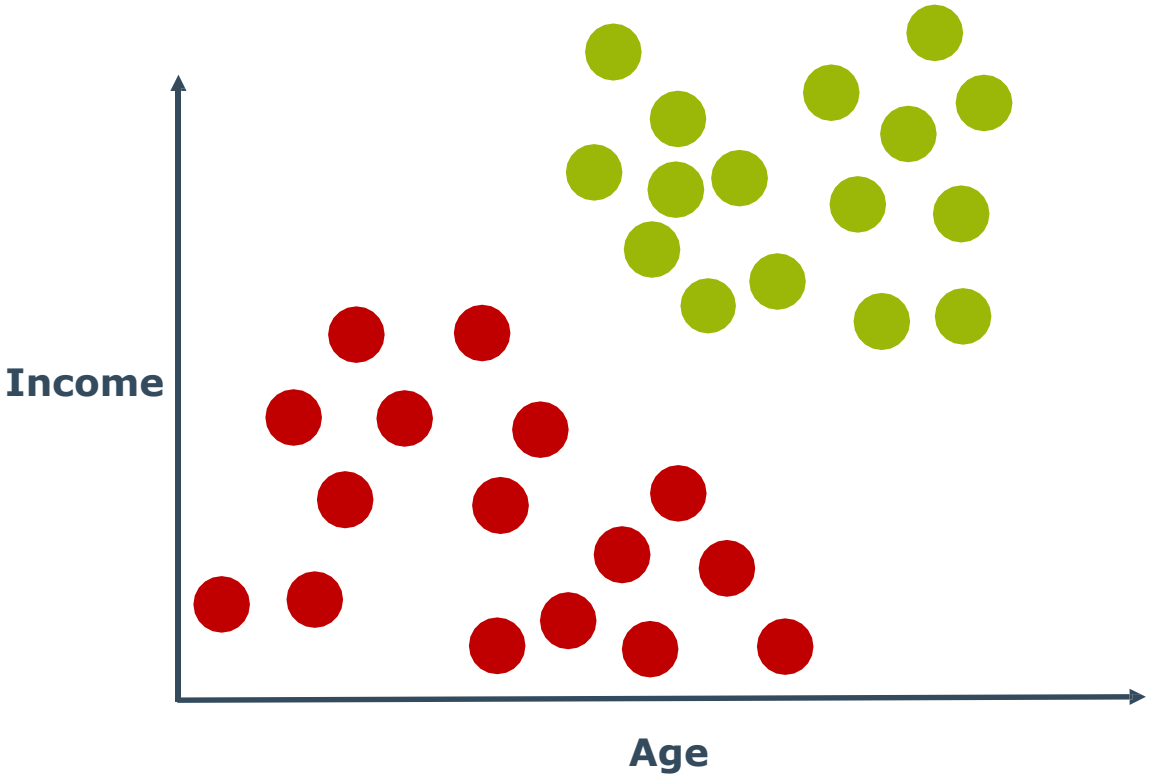
聚合式层次聚类

当前聚簇数 = 3



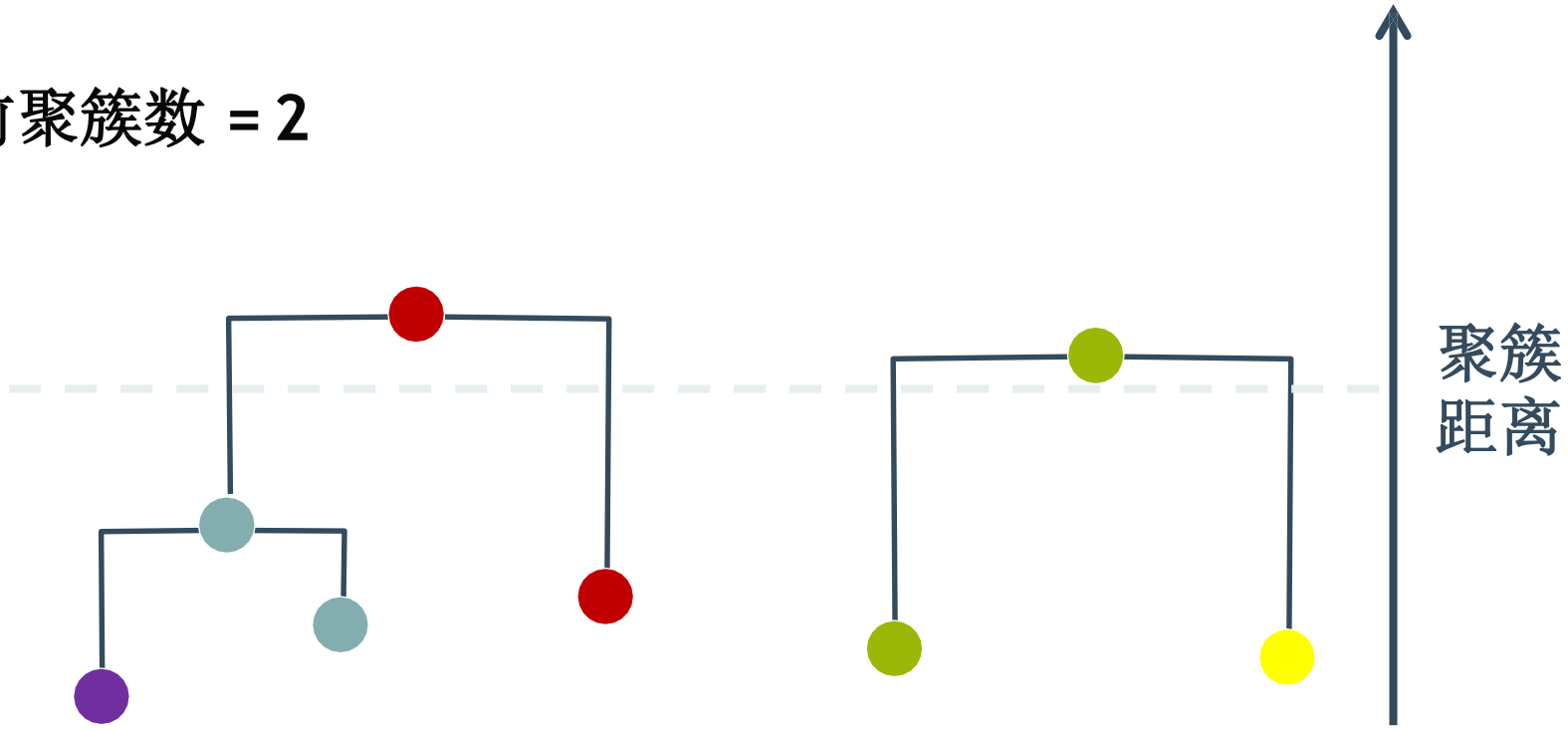
聚合式层次聚类

当前聚簇数 = 2



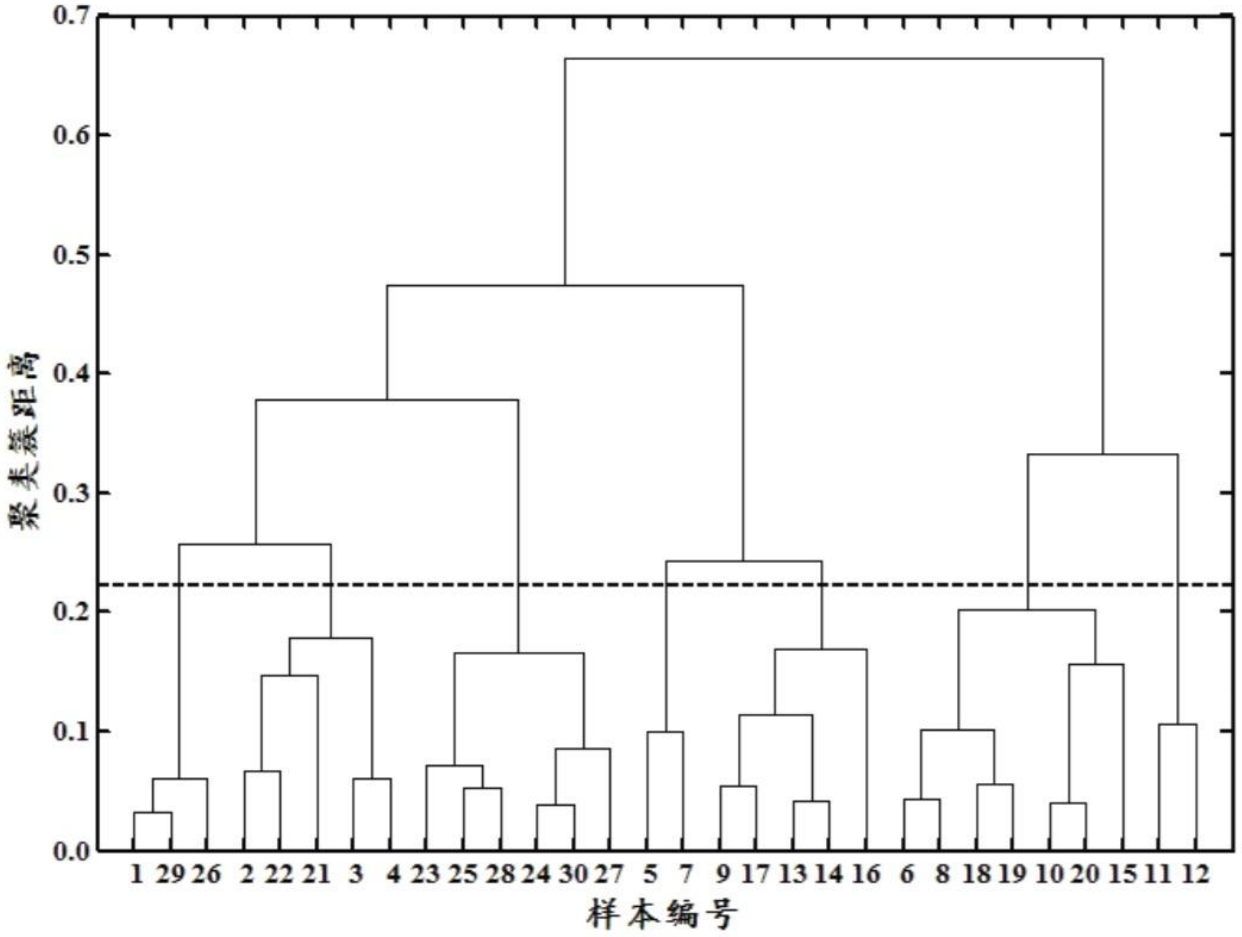
聚合式层次聚类

当前聚簇数 = 2



树状图

AGNES算法树状图：



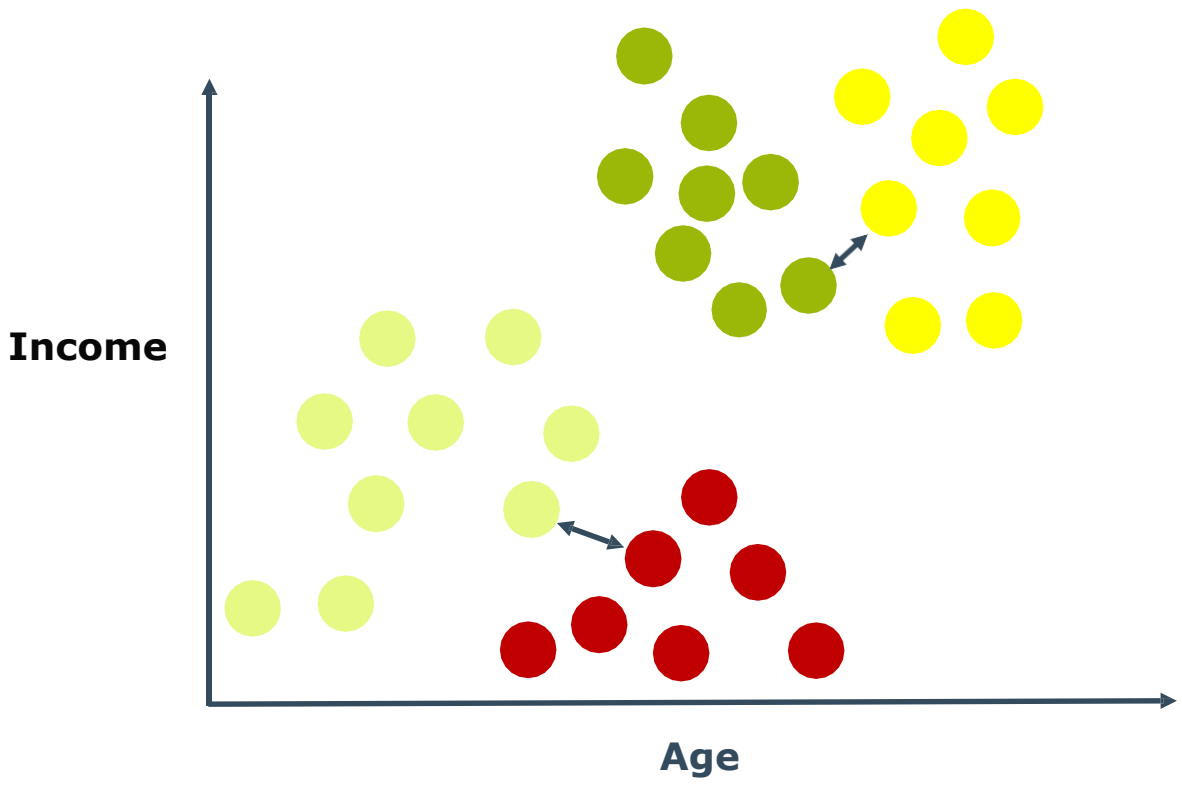
簇间距离类型

簇间距离的类型

- **单链接**（Single Linkage）：簇间最小的两点距离
- **完全链接**（Complete Linkage）：簇间最大的两点距离
- **平均链接**（Average Linkage）：簇间所有两点距离的平均值
- **Ward链接**（Ward Linkage）：每次选择导致最佳inertia的合并

簇间距离的类型

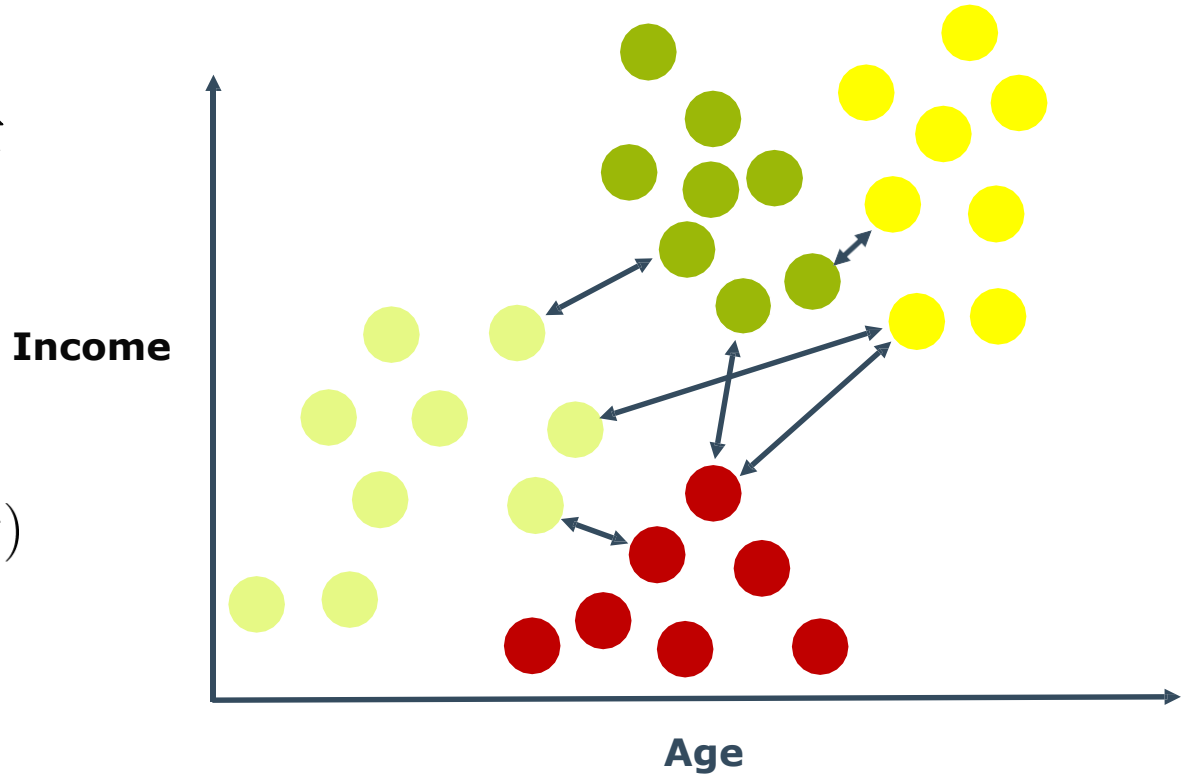
单链接：簇间最小的两点距离



簇间距离的类型

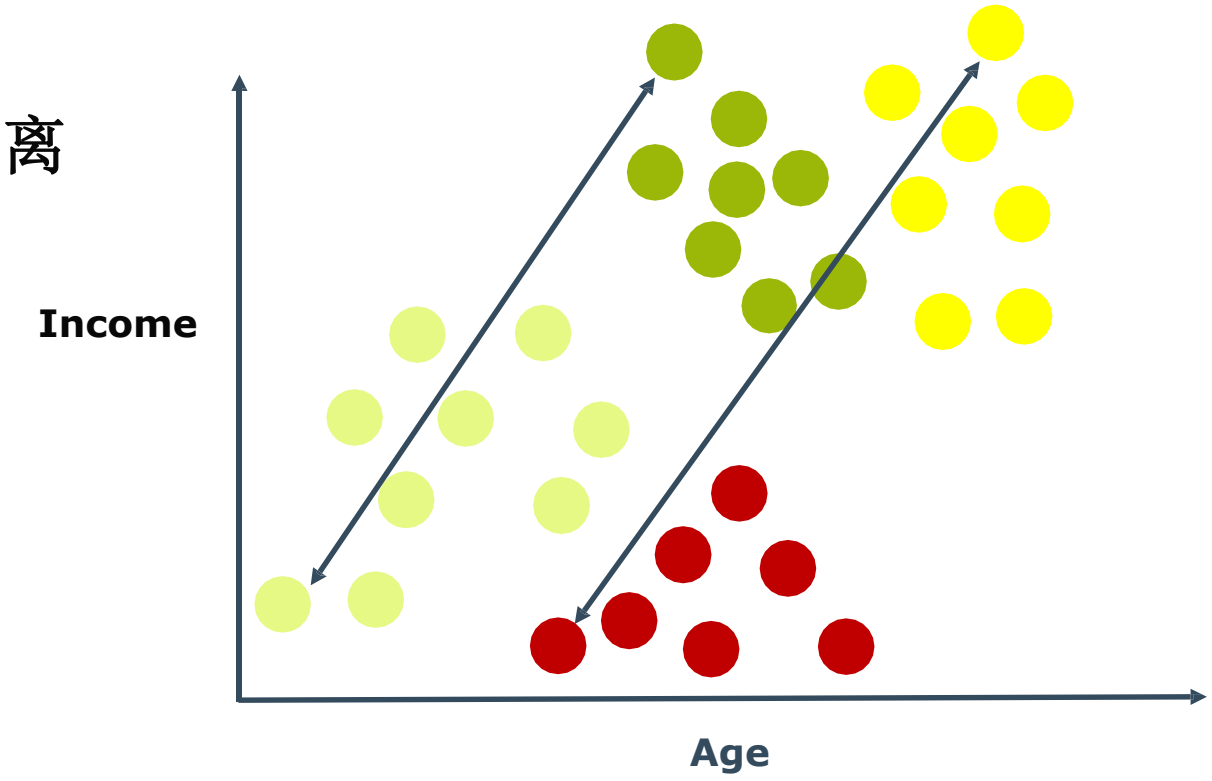
单链接：簇间最小的两点距离

$$d_{min}(C_i, C_j) = \min_{x \in C_i, z \in C_j} dist(x, z)$$



簇间距离的类型

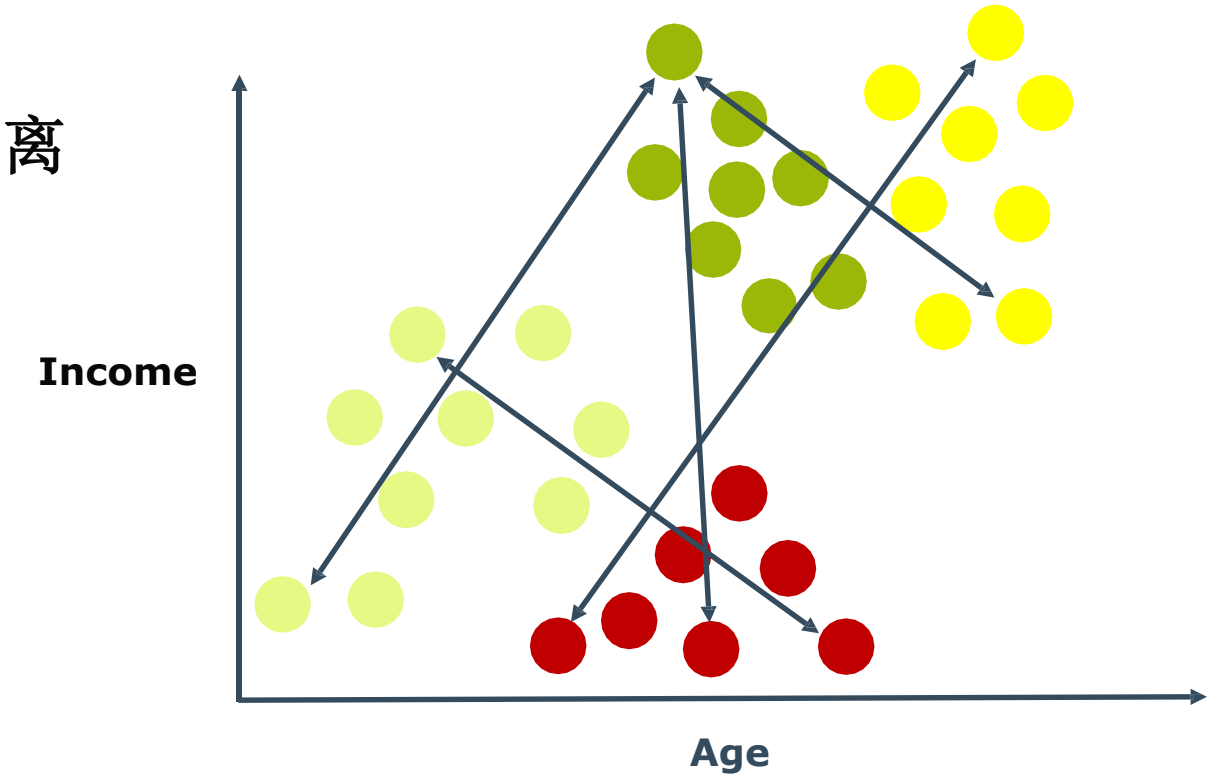
完全链接：簇间最大的两点距离



簇间距离的类型

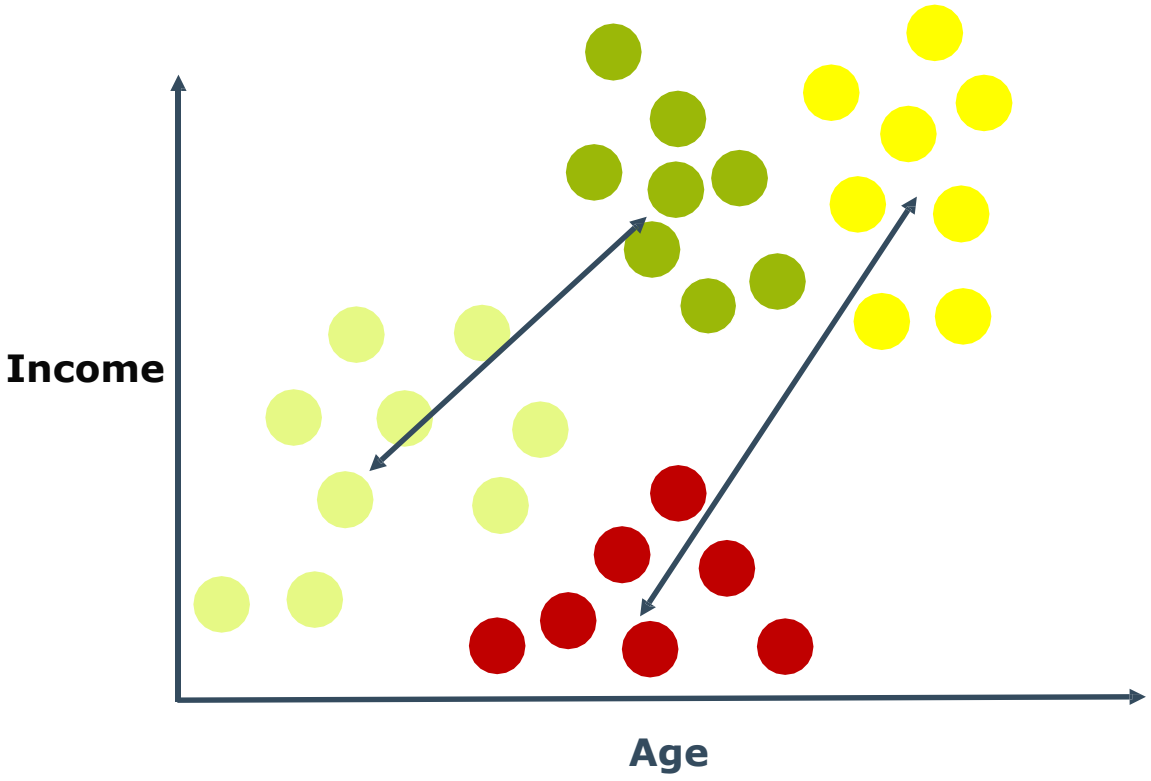
完全链接：簇间最大的两点距离

$$d_{max}(C_i, C_j) = \max_{x \in C_i, z \in C_j} dist(x, z)$$



簇间距离的类型

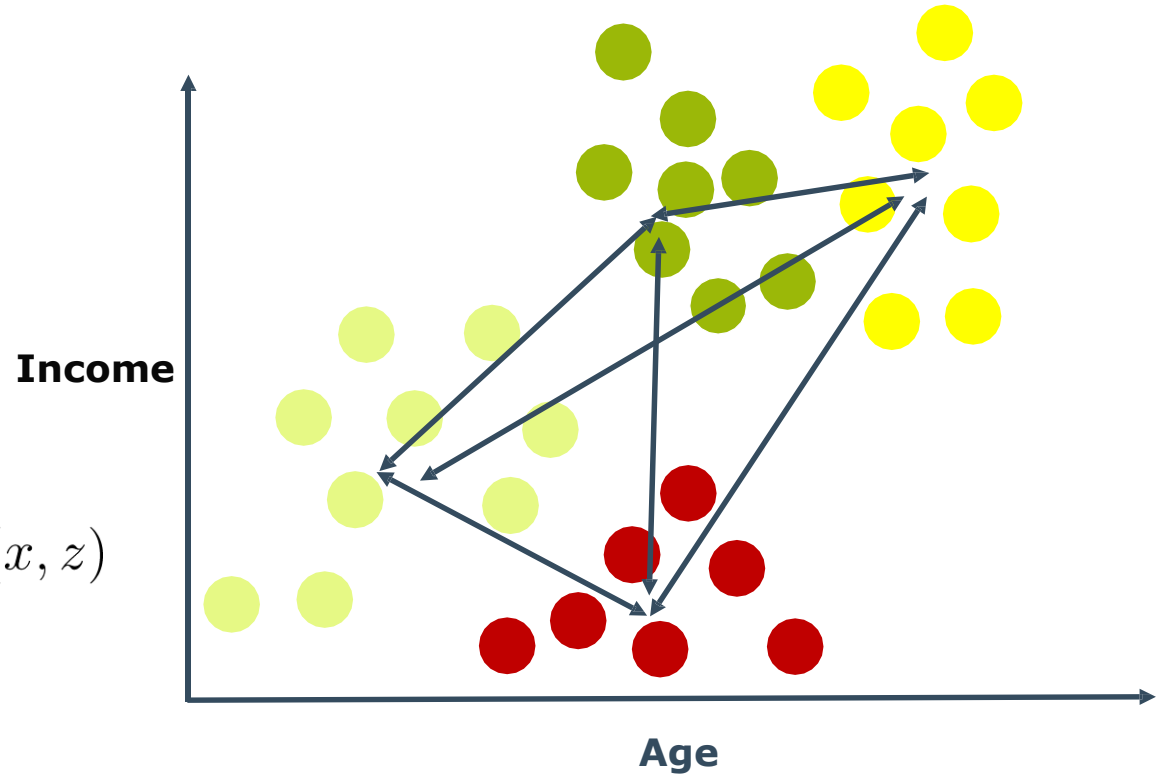
平均链接：
簇间所有两点距离的平均值



簇间距离的类型

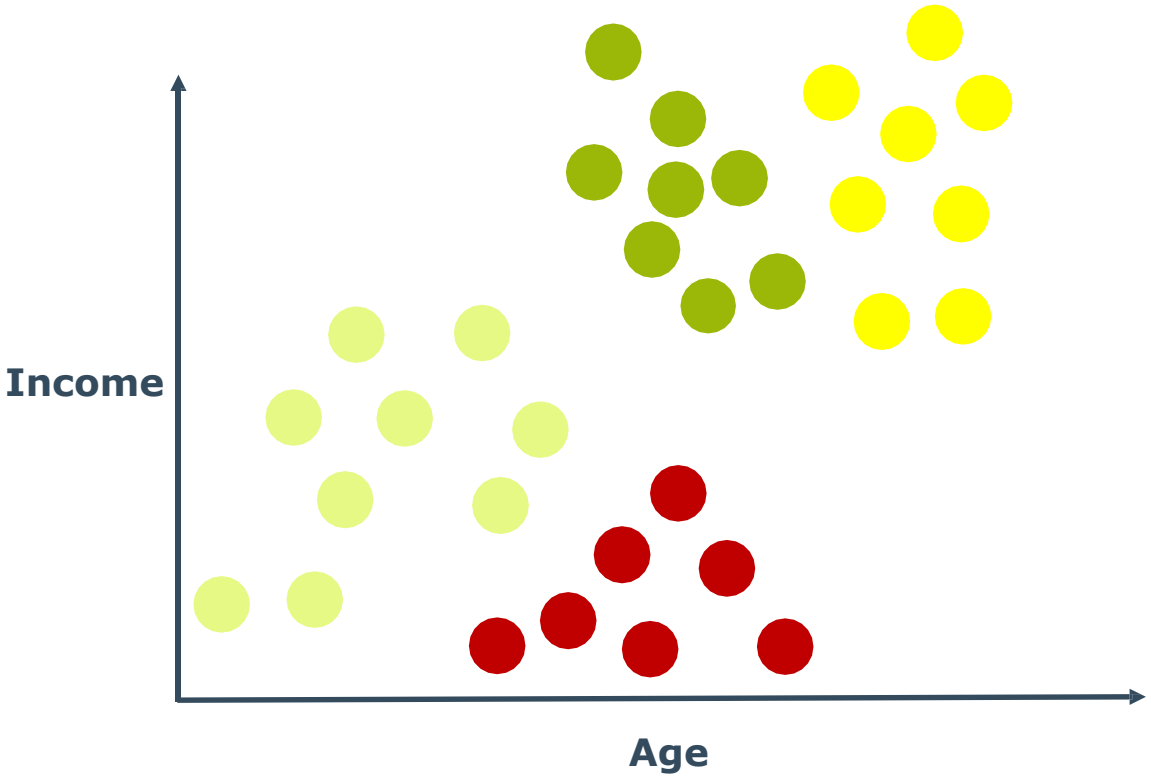
平均链接：
簇间所有两点距离的平均值

$$d_{avg}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{z \in C_j} dist(x, z)$$



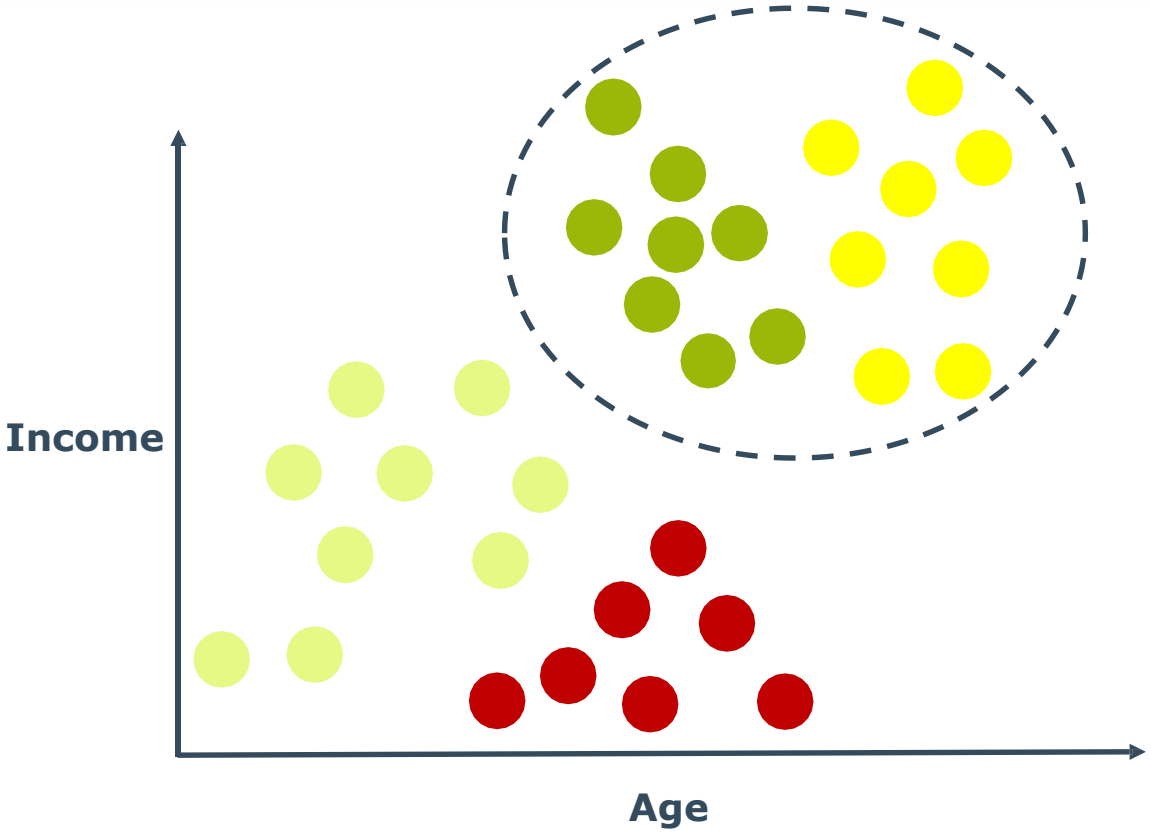
簇间距离的类型

Ward链接：每次选择导致最佳inertia的合并



簇间距离的类型

Ward链接：每次选择导致最佳inertia的合并



聚合式层次聚类的语法

导入包含聚类方法的类：

```
from sklearn.cluster import AgglomerativeClustering
```

创建该类的一个对象：

```
agg = AgglomerativeClustering(n_clusters=3,
                              affinity='euclidean',
                              linkage='ward')
```

拟合数据，并预测新数据的聚簇：

```
agg = agg.fit(X1)
y_predict = agg.predict(X2)
```

http://scikit-learn.org/stable/modules/generated/sklearn.cluster.AgglomerativeClustering.html

聚合式层次聚类的语法

导入包含聚类方法的类：

```
from sklearn.cluster import AgglomerativeClustering
```

创建该类的一个对象：

```
agg = AgglomerativeClustering(n_clusters=3,
                              affinity='euclidean',
                              linkage='ward')
```

最终的聚簇数

拟合数据，并预测新数据的聚簇：

```
agg = agg.fit(X1)
y_predict = agg.predict(X2)
```

聚合式层次聚类的语法

导入包含聚类方法的类：

```
from sklearn.cluster import AgglomerativeClustering
```

创建该类的一个对象：

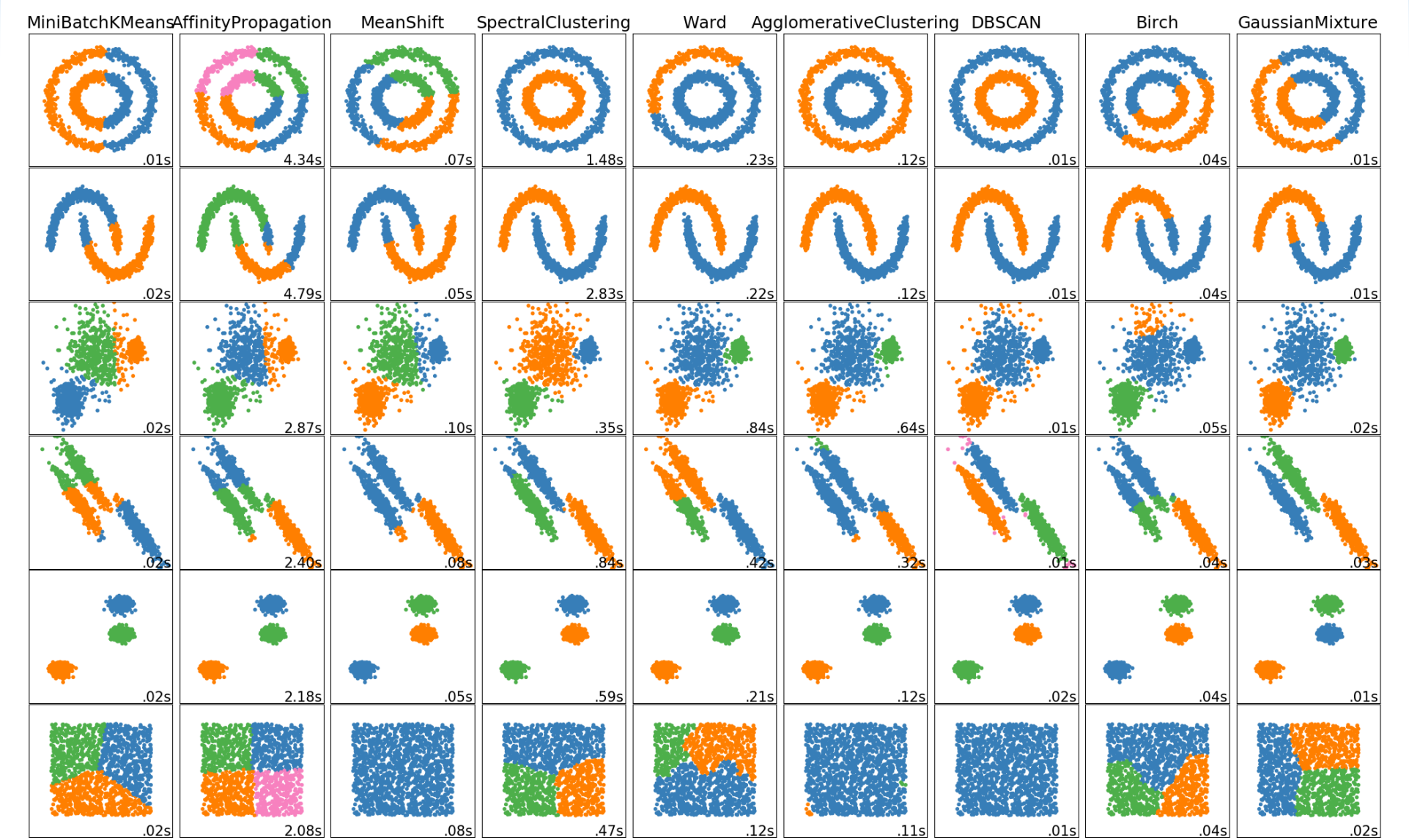
```
agg = AgglomerativeClustering(n_clusters=3,
                              affinity='euclidean',
                              linkage='ward')
```

聚簇间距离的计算和聚合方法

拟合数据，并预测新数据的聚簇：

```
agg = agg.fit(X1)
y_predict = agg.predict(X2)
```

其他聚类算法



Reference: http://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

- 在商业上被用来发现不同的客户群，并且通过购买模式刻画不同客户群的特征。聚类分析是细分市场的有效工具，同时也是用于研究消费者行为，寻找新的潜在市场。
- 在生物学上，聚类能用于推导植物和动物的分类，对基因进行分类，获得对种群中固有结构的认识。
- 在线文档的快速发展，文本聚类研究成为关注的重点。对给定文本，需要根据它们内容的相似性来进行组织。建立一个主题层次。